

UNIVERSIDAD POLITÉCNICA DE MADRID
Escuela Técnica Superior de Ingeniería de Sistemas
Informáticos



UNIVERSIDAD
POLITÉCNICA
DE MADRID



Máster
Deep Learning
Universidad Politécnica de Madrid

Trabajo Fin de Máster

**Sistema LLM para análisis financiero histórico
mediante generación automática de código**

MÁSTER DE FORMACIÓN PERMANENTE EN DEEP LEARNING

Presentado por:

Carlos Ruiz Oyarzun

Bajo la dirección de:

Adrián Girón Jiménez

Alejandro Delgado Peribáñez

Madrid, 2026

Resumen

Este trabajo presenta el diseño y desarrollo de un sistema basado en modelos de lenguaje para realizar análisis financiero histórico a partir de consultas formuladas en lenguaje natural. El objetivo no es predecir la evolución del mercado, sino estudiar cómo integrar un LLM en un flujo de trabajo en el que los cálculos puedan ejecutarse, revisarse y conservarse como evidencia.

El sistema recibe una consulta junto con información estructurada sobre los activos, el periodo analizado y los ficheros CSV disponibles. A partir de esta entrada, un primer agente elabora un plan analítico; un segundo agente genera el código Python necesario; y un tercer agente interpreta los resultados calculados. Entre estas etapas se aplican validaciones de entrada, controles sobre el código generado y una ejecución en un proceso separado. El flujo conserva el plan, el script, la salida estructurada y los registros de error, lo que permite localizar en qué etapa se produce un fallo.

Para comprobar el funcionamiento del sistema se ha definido una batería de consultas con distintos niveles de dificultad y requisitos analíticos. La evaluación contempla tanto la finalización del flujo como la calidad de las respuestas, la correcta utilización de los datos y la gestión controlada de los errores. De este modo, el trabajo propone una aproximación reproducible para estudiar la integración de modelos de lenguaje, generación de código y análisis financiero, manteniendo la trazabilidad del proceso y evitando la formulación de recomendaciones de inversión.

Palabras clave: análisis financiero histórico, modelos de lenguaje, generación de código, agentes LLM, ejecución controlada, trazabilidad.

Abstract

This work presents the design and development of a language-model-based system for historical financial analysis from natural-language queries. Its purpose is not to predict market behaviour, but to examine how an LLM can be integrated into a workflow in which calculations are executed, reviewed and retained as evidence.

The system receives a query together with structured information about the requested assets, time period and available CSV files. A first agent produces an analytical plan, a second agent generates the required Python code, and a third agent explains the calculated results. Input validation, code checks and execution in a separate process are applied between these stages. The workflow retains the plan, generated script, structured output and error logs, making it possible to identify the stage at which a failure occurs.

To examine the operation of the system, an evaluation set was defined with queries covering different levels of difficulty and analytical requirements. The assessment considers workflow completion, response quality, appropriate use of the available data and controlled error handling. The work therefore provides a reproducible approach for studying the integration of language models, code generation and financial analysis while preserving process traceability and avoiding investment recommendations.

Keywords:

historical financial analysis, language models, code generation, LLM agents, traceability.

Índice general

Resumen	I
Abstract	II
1. Introducción	1
1.1. Contexto y definición del problema	1
1.2. Motivación	2
1.3. Objetivos	3
1.4. Alcance	4
1.5. Estructura de la memoria	4
2. Estado de la cuestión	5
2.1. Enfoque de la revisión	5
2.2. Modelos de lenguaje y agentes financieros	5
2.3. Agentes, herramientas y orquestación	6
2.4. Generación y ejecución controlada de código	7
2.5. Fiabilidad y evaluación de sistemas generativos	8
2.6. Comparativa y posicionamiento de la propuesta	8
3. Datos y material de trabajo	10
3.1. Alcance del capítulo	10
3.2. Procedencia y obtención de los datos	10
3.3. Inventario y cobertura	11
3.4. Estructura y normalización	12
3.5. Análisis exploratorio y calidad	13
3.6. Batería de consultas	14
3.7. Reproducibilidad y limitaciones	15
4. Metodología	16
4.1. Enfoque metodológico	16
4.1.1. Preguntas de evaluación	16
4.2. Proceso de desarrollo	17

4.2.1.	Verificación previa del prototipo	18
4.3.	Diseño de la evaluación	18
4.3.1.	Unidad de análisis	18
4.3.2.	Batería de consultas	18
4.4.	Protocolo de ejecución	19
4.5.	Cadena de validación	20
4.6.	Métricas de evaluación	21
4.6.1.	Indicadores técnicos	21
4.6.2.	Calidad de la respuesta	22
4.6.3.	Protocolo de revisión humana	23
4.7.	Tratamiento de los resultados	23
4.8.	Reproducibilidad y limitaciones metodológicas	24
5.	Diseño del sistema	26
5.1.	Objetivos y principios de diseño	26
5.2.	Arquitectura general	27
5.3.	Entrada y estado compartido	29
5.3.1.	Entrada estructurada	29
5.3.2.	Estado del flujo	30
5.4.	Etapas del flujo	30
5.4.1.	Ingesta y validación	30
5.4.2.	Agente 1: planificación analítica	30
5.4.3.	Agente 2: generación del análisis	31
5.4.4.	Validación y reparación	31
5.4.5.	Ejecución controlada	31
5.4.6.	Agente 3: interpretación	32
5.5.	Rutas de error y recuperación	32
5.6.	Trazabilidad y conservación de evidencias	33
5.7.	Decisiones de diseño y alcance	34
6.	Implementación	36
6.1.	Organización técnica de la implementación	36
6.2.	Orquestación y gestión del estado	37
6.3.	Contratos y validación de entrada	38
6.4.	Carga y normalización de los datos	39
6.5.	Ejecución controlada y tratamiento de resultados	40
6.6.	Validación, reparación y gestión de errores	41
6.7.	Configuración, trazabilidad y reproducibilidad	42
6.8.	Pruebas automatizadas	42

7. Integración del modelo de lenguaje	44
7.1. Papel del modelo y división de responsabilidades	44
7.2. Diseño común de los prompts	45
7.3. Prompts específicos de los agentes	46
7.3.1. Agente 1: planificación analítica	46
7.3.2. Agente 2: generación de código	47
7.3.3. Reparación del código generado	47
7.3.4. Agente 3: interpretación de resultados	48
7.4. Niveles de profundidad A, B y C	48
7.5. Procesamiento de respuestas y gestión de errores	49
7.6. Cliente, modelo y configuración	50
7.7. Alcance de la estrategia de prompts	50
8. Resultados y evaluación	52
8.1. Objetivo	52
8.2. Batería A/B/C	52
8.3. Métricas técnicas	53
8.4. Resultados observados	53
8.5. Limitaciones	54
8.6. Síntesis	54
9. Aspectos éticos, seguridad y uso responsable	55
9.1. Enfoque general	55
9.2. Finalidad prevista y usos excluidos	55
9.3. Interpretación financiera y supervisión humana	56
9.4. Calidad, procedencia y alcance de los datos	57
9.5. Seguridad del código generado	58
9.6. Trazabilidad y rendición de cuentas	59
9.7. Privacidad, confidencialidad y credenciales	59
9.8. Coste computacional y sostenibilidad	59
9.9. Condiciones para un uso futuro	60
10. Conclusiones y trabajo futuro	61
Referencias	62
A. Anexos	66
A.1. Ejecución de la batería	66
A.1.1. Ejecución completa	66
A.1.2. Ejecución por niveles	67
A.1.3. Ejecución en bloques o por casos	67

A.2. Configuración de API	68
A.3. Bateria de consultas A/B/C	69
A.4. Plantillas de prompts de los agentes	71
A.4.1. Instrucción común	71
A.4.2. Agente 1: planificación analítica	72
A.4.3. Agente 2: generación de código	72
A.4.4. Reparación del código	73
A.4.5. Agente 3: interpretación	74
A.5. Trazabilidad de una ejecución	75
A.6. Rúbrica de evaluación cualitativa	75

Índice de tablas

2.1. Comparación entre los trabajos relacionados y la propuesta desarrollada.	9
3.1. Inventario y cobertura temporal de los datos utilizados.	11
3.2. Campos financieros presentes en los CSV.	12
3.3. Resumen de cobertura y calidad del conjunto de datos.	13
3.4. Distribución de la batería de 50 consultas por nivel.	14
4.1. Cadena de validación aplicada a cada ejecución.	21
5.1. Responsabilidades principales de la arquitectura.	29
5.2. Principales rutas de error y recuperación del sistema.	33
5.3. Información conservada para garantizar la trazabilidad de cada caso. .	34
6.1. Bloques funcionales de la implementación.	37
6.2. Evolución simplificada del estado durante una ejecución.	38
6.3. Controles aplicados antes de la ejecución.	41
7.1. Diferencias principales entre los tres agentes.	45
7.2. Adaptación del comportamiento según el nivel de profundidad.	49
8.1. Estructura de la batería A/B/C de 15 consultas.	52
8.2. Lectura de los estados principales de evaluación.	54
9.1. Delimitación del uso responsable del sistema.	56
9.2. Controles de seguridad y riesgos que permanecen abiertos.	58
A.1. Distribución de la batería de 50 consultas.	69
A.2. Batería completa de 50 consultas por nivel de profundidad.	69
A.3. Información conservada para revisar cada caso de la batería.	75
A.4. Métricas cualitativas aplicadas a los casos de evaluación.	76

Índice de figuras

1.1. Mapa conceptual de relaciones entre inteligencia artificial, aprendizaje automático, redes neuronales, aprendizaje profundo e inteligencia artificial generativa. Fuente: Analytics Vidhya [2].	2
5.1. Flujo resumido del sistema desde la consulta del usuario hasta la respuesta final.	28

Capítulo 1

Introducción

1.1. Contexto y definición del problema

Los modelos de lenguaje de gran escala han ampliado de forma notable las posibilidades de interacción con los sistemas informáticos. Además de producir texto, pueden interpretar instrucciones, proponer procedimientos y generar código ejecutable. Buena parte de este avance se apoya en la arquitectura Transformer, presentada en *Attention Is All You Need* [1], cuyo mecanismo de atención constituye una de las bases de los modelos de lenguaje actuales.

La capacidad de generar código resulta especialmente interesante en tareas de análisis de datos. Una consulta expresada en lenguaje natural puede transformarse en operaciones de carga, filtrado, cálculo y presentación de resultados. Sin embargo, esta flexibilidad introduce un problema de confianza: que el código sea plausible no significa que sea correcto, y que una respuesta esté bien redactada no garantiza que respete los datos de partida.

Este problema cobra especial importancia en el ámbito financiero. Incluso cuando el análisis se limita a información histórica, una cifra incorrecta o una interpretación poco prudente puede conducir a conclusiones equivocadas. Por ello, no basta con pedir al modelo que responda directamente a una consulta: también es necesario conocer qué datos ha utilizado, qué métricas ha calculado, qué código ha ejecutado y qué limitaciones presenta el resultado.

Este trabajo aborda el problema desde la integración de sistemas, no desde el entrenamiento de un nuevo modelo. Para ello, se desarrolla un prototipo que recibe una consulta financiera y un conjunto de datos históricos almacenados en CSV, organiza el análisis mediante agentes LLM y mantiene los cálculos dentro de un flujo programático. El sistema separa la planificación, la generación de código y la interpretación final, valida el script antes de ejecutarlo y conserva los principales artefactos de cada ejecución.

La Figura 1.1 sitúa el trabajo dentro de las principales áreas de la inteligencia artificial y muestra la relación entre aprendizaje automático, aprendizaje profundo e inteligencia artificial generativa. El proyecto se enmarca en esta última área, aunque su aportación no consiste en desarrollar un modelo generativo, sino en estudiar su uso dentro de una arquitectura trazable de análisis financiero histórico.

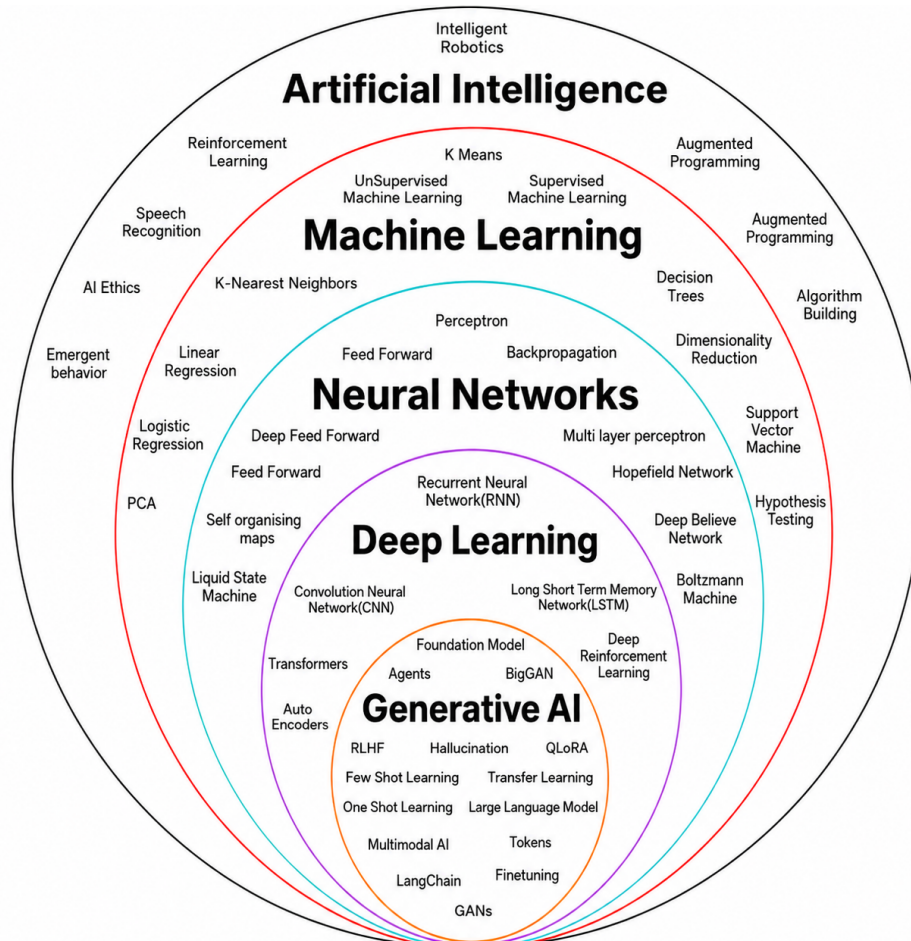


Figura 1.1: Mapa conceptual de relaciones entre inteligencia artificial, aprendizaje automático, redes neuronales, aprendizaje profundo e inteligencia artificial generativa. Fuente: Analytics Vidhya [2].

1.2. Motivación

El proyecto nace del interés por combinar tres ámbitos que a menudo se estudian por separado: los modelos de lenguaje, la programación para el análisis de datos y la interpretación financiera. Los LLM permiten construir con rapidez interfaces capaces de entender peticiones diversas, pero esa misma facilidad puede ocultar errores cuando todo el proceso queda reducido a una única respuesta generativa.

La motivación principal es comprobar hasta qué punto puede aprovecharse la flexibilidad del modelo sin renunciar a mecanismos básicos de control. En el sistema

propuesto, el LLM decide cómo abordar la consulta y genera el código necesario, pero el resultado financiero procede de la ejecución de ese código sobre datos locales. Esta separación permite revisar tanto el planteamiento seguido como los cálculos obtenidos.

Otro criterio importante es que los fallos sean visibles. Si el prototipo no puede generar código válido, la ejecución no termina correctamente o la salida supera las capacidades del modelo, el sistema registra el problema y devuelve un error controlado. Este comportamiento resulta preferible a completar una respuesta con cifras no verificadas. La evaluación mediante 50 consultas permite observar estos fallos, especialmente en los casos de mayor dificultad, e identificar en qué parte del proceso se producen.

En esta memoria, el término MVP hace referencia a un prototipo mínimo viable construido de forma incremental, no a un producto financiero listo para su uso. Las primeras iteraciones se centraron en consultas sencillas de crecimiento y comparación. Más adelante se incorporaron nuevos tipos de análisis, tres agentes diferenciados, mecanismos de validación y reparación del código, la ejecución en un proceso separado y la conservación de artefactos. Esta evolución se describe en la Sección 4.2.

1.3. Objetivos

El objetivo general del trabajo es diseñar, implementar y evaluar un sistema basado en un modelo de lenguaje capaz de transformar consultas sobre datos financieros históricos en cálculos ejecutables y respuestas trazables.

Los objetivos específicos son:

- Diseñar una arquitectura modular que separe la planificación, la generación de código, su validación, la ejecución de los cálculos y la elaboración de la respuesta final.
- Desarrollar un flujo capaz de interpretar consultas con diferentes requisitos analíticos y determinar los datos, métricas y operaciones necesarios.
- Generar y ejecutar código Python adaptado a cada consulta, aplicando controles que limiten las operaciones permitidas.
- Garantizar la trazabilidad del proceso mediante la conservación del plan, el código generado, los resultados estructurados y los registros de error.
- Generar respuestas financieras basadas exclusivamente en los cálculos realizados, sin incorporar predicciones ni recomendaciones de inversión.
- Evaluar la capacidad del modelo de lenguaje para comprender y adaptarse a las tareas planteadas, generar código acorde con sus requisitos y redactar respuestas claras, precisas y fundamentadas.

- Analizar el comportamiento global del sistema mediante una batería de 50 consultas de distinta complejidad, pruebas automatizadas y métricas de evaluación.

1.4. Alcance

El sistema se limita al análisis histórico y descriptivo de activos financieros a partir de datos descargados previamente y almacenados en ficheros CSV. Los casos utilizados incluyen acciones, índices, fondos cotizados, divisas, una materia prima y un criptoactivo, con diferentes periodos e intervalos temporales.

El prototipo no obtiene precios en tiempo real durante la ejecución, no incorpora noticias ni información macroeconómica y tampoco realiza análisis fundamental. No pretende predecir movimientos futuros ni recomendar la compra o venta de activos. Estas restricciones responden al propósito del trabajo: evaluar una arquitectura LLM trazable, no construir un asesor financiero.

La validación de seguridad limita determinadas operaciones mediante el análisis del árbol sintáctico del código, aunque no equivale a un entorno aislado de producción. Por otra parte, la batería de 50 casos permite observar el comportamiento del sistema ante un conjunto amplio de consultas, pero una única ejecución no basta para establecer su robustez estadística frente a la variabilidad propia de los modelos generativos.

1.5. Estructura de la memoria

La memoria se organiza siguiendo el recorrido del proyecto. El Capítulo 2 revisa los trabajos relacionados con modelos de lenguaje en finanzas, generación de código y trazabilidad. El Capítulo 3 describe los datos empleados y el análisis exploratorio realizado, mientras que el Capítulo 4 presenta la metodología y los criterios de evaluación.

Los Capítulos 5 y 6 exponen la arquitectura y su implementación, incluidos los contratos de datos, la validación del código y el proceso de ejecución. El Capítulo 7 se centra en la integración de los tres agentes LLM. El Capítulo 8 presenta la batería de 50 consultas, los resultados y los principales fallos observados. Finalmente, el Capítulo 9 aborda el uso responsable y la seguridad; el Capítulo 10 recoge las conclusiones y el trabajo futuro; y los anexos reúnen la configuración, las consultas, los prompts y distintas trazas de ejecución.

Capítulo 2

Estado de la cuestión

2.1. Enfoque de la revisión

Este capítulo sitúa el trabajo dentro del uso de modelos de lenguaje para el análisis financiero y la automatización de tareas de datos. No se pretende revisar todo el campo de la inteligencia artificial aplicada a las finanzas, sino centrarse en las líneas que sustentan el sistema desarrollado: los modelos especializados en este dominio, las arquitecturas basadas en agentes, el uso de herramientas externas, la generación de código y la evaluación de sistemas generativos.

El punto de partida es un modelo ya disponible, integrado en un flujo de trabajo controlado. Ante una consulta en lenguaje natural, el modelo interviene en la planificación del análisis, la generación de código Python y la redacción de una respuesta basada en los resultados obtenidos al analizar datos históricos locales. Quedan fuera del alcance el entrenamiento de un modelo propio, la predicción de precios y la toma autónoma de decisiones de inversión.

Esta delimitación permite orientar la revisión hacia trabajos que combinan lenguaje natural, cálculo programático y mecanismos de control. Como responden a problemas y escalas diferentes, no tendría sentido comparar directamente sus resultados. Lo relevante es identificar las ideas que ayudan a justificar las decisiones adoptadas en el prototipo.

2.2. Modelos de lenguaje y agentes financieros

La aplicación del procesamiento del lenguaje natural al ámbito financiero es anterior a los modelos generativos actuales. FinBERT adapta la arquitectura BERT al lenguaje financiero y muestra la utilidad de los modelos especializados en tareas como el análisis de sentimiento [3]. Posteriormente, BloombergGPT y FinGPT ampliaron esta línea mediante modelos entrenados o adaptados con noticias, documentación corporativa y

otros recursos propios del sector [4], [5].

Estos trabajos muestran que el lenguaje financiero contiene vocabulario y relaciones que requieren una interpretación cuidadosa. Sin embargo, su objetivo principal se centra en construir o adaptar el modelo. Aquí se adopta una perspectiva diferente: se utiliza `openai/gpt-oss-20b`, un modelo de pesos abiertos, y se estudia su comportamiento dentro de un sistema verificable de análisis histórico [6]. La atención se desplaza así del entrenamiento a la integración, el control y la trazabilidad.

La evolución de los LLM financieros también ha favorecido la aparición de sistemas basados en agentes. FinRobot propone una arquitectura orientada a la investigación y valoración de empresas [7], mientras que TradingAgents distribuye tareas de análisis, debate y toma de decisiones de trading entre varios agentes [8]. En ambos casos, el modelo deja de actuar únicamente como generador de texto y pasa a intervenir en un proceso compuesto por distintas responsabilidades.

El sistema comparte esta separación funcional, aunque limita deliberadamente la autonomía. Un mismo modelo desempeña tres funciones: interpretar la consulta, generar el código y explicar los resultados. No se simula una organización de analistas, no se permite actuar sobre el mercado ni se producen recomendaciones de inversión. Esta delimitación facilita la supervisión de cada fase y permite estudiar por separado la comprensión de la tarea, la capacidad de generar código y la calidad de la redacción final.

2.3. Agentes, herramientas y orquestación

Los sistemas basados en agentes permiten que un modelo vaya más allá de la generación de texto: puede utilizar herramientas y trabajar con los resultados de acciones externas. ReAct propone alternar razonamiento y actuación para que el modelo observe las consecuencias de sus decisiones [9]. Toolformer estudia cómo puede aprender a recurrir a herramientas cuando las necesita [10]. Las revisiones sobre agentes basados en LLM describen esta evolución como el paso de sistemas puramente conversacionales a arquitecturas capaces de planificar, actuar y utilizar observaciones del entorno [11].

Para coordinar estas tareas es necesario conservar la información producida durante la ejecución. LangGraph representa los flujos mediante un estado compartido, nodos que realizan operaciones y transiciones que determinan el siguiente paso [12]. Esta estructura encaja bien cuando se combinan componentes generativos con validaciones y decisiones programáticas.

El sistema adopta este patrón, pero sigue un recorrido definido de antemano. El modelo no selecciona libremente las herramientas ni modifica el orden del proceso. Cada consulta pasa por las fases de planificación, generación de código, validación, ejecución e interpretación. Si aparece un error, las transiciones determinan si se intenta

una reparación o si el flujo debe finalizar de forma controlada.

El estado compartido funciona como una memoria de trabajo limitada a cada ejecución. Conserva las entradas y salidas de las distintas fases, pero no mantiene una memoria conversacional permanente. De este modo, se puede revisar el recorrido completo de cada caso sin incorporar información de consultas anteriores. También resulta más fácil localizar el punto exacto en el que se produjo un fallo, aunque la respuesta final parezca convincente.

2.4. Generación y ejecución controlada de código

La generación automática de código es una de las capacidades que ha impulsado el uso de los modelos de lenguaje en tareas técnicas. El trabajo sobre Codex y HumanEval mostró que un modelo entrenado con código puede sintetizar programas a partir de descripciones en lenguaje natural [13]. En ciencia de datos, DS-1000 propuso un banco de pruebas con problemas realistas y librerías habituales de análisis [14].

Estas investigaciones respaldan el uso de un LLM para traducir una consulta financiera en operaciones de carga, filtrado y cálculo. También ponen de manifiesto una limitación fundamental: que un programa parezca correcto no implica que sea ejecutable, seguro o fiel a la petición. Por ello, el código no se considera una respuesta final, sino un artefacto intermedio que debe superar varios controles.

La comunicación entre el modelo y la aplicación se apoya en salidas estructuradas. El plan analítico, el código generado y los resultados de la ejecución siguen contratos JSON conocidos, mientras que el texto libre se reserva para la explicación destinada al usuario. Esta separación reduce la ambigüedad y permite detectar respuestas incompletas o mal formadas antes de pasar a la siguiente fase.

Ejecutar código generado por un modelo introduce riesgos específicos. Los estudios sobre asistentes de programación han mostrado que sus sugerencias pueden contener vulnerabilidades, incluso cuando parecen funcionales [15]. Por este motivo, el código debe validarse antes de su ejecución y limitarse a un conjunto restringido de operaciones.

El prototipo combina revisión estática, limitación de importaciones, un proceso de ejecución separado y un tiempo máximo. Estas medidas reducen la superficie de riesgo, aunque no equivalen a una sandbox completa. En un entorno de producción sería necesario añadir más aislamiento, permisos mínimos y límites de recursos.

Los resultados numéricos se obtienen al ejecutar el código sobre los CSV locales y se conservan en una salida estructurada. El agente encargado de redactar la respuesta recibe esos valores, pero no debe sustituirlos por cifras generadas libremente. Así se mantiene una separación clara entre el cálculo y su comunicación.

2.5. Fiabilidad y evaluación de sistemas generativos

Una limitación conocida de los modelos generativos es la producción de respuestas plausibles que no se corresponden con la información disponible, fenómeno denominado habitualmente alucinación [16]. La generación aumentada con recuperación busca reducir este problema apoyando la respuesta en fuentes obtenidas de forma explícita [17]. Aunque el sistema no incorpora recuperación documental, sigue un principio de anclaje similar: la explicación final debe basarse en datos locales, código ejecutado y resultados registrados.

Este anclaje es especialmente importante en el análisis financiero. Cuando una respuesta presenta datos de rentabilidad, volatilidad o drawdown, debe ser posible revisar tanto la información de partida como el procedimiento seguido. También conviene separar con claridad la descripción del comportamiento histórico de cualquier predicción o recomendación. Conservar la consulta, el plan, el código, los resultados, los errores y la respuesta final no elimina la posibilidad de fallo, pero ayuda a localizarlo y valorar sus consecuencias.

La evaluación de un sistema de estas características no puede reducirse a una única cifra. HELM defiende una evaluación amplia y transparente, compuesta por distintos escenarios y métricas, para evitar que un resultado agregado oculte capacidades y limitaciones relevantes [18]. En el caso del texto generado, G-Eval señala además que las métricas automáticas tradicionales no siempre reflejan la calidad percibida durante una evaluación humana [19].

Siguiendo estas ideas, la evaluación distingue entre el funcionamiento técnico y la calidad de la respuesta. La parte técnica comprueba si el flujo genera un plan, produce código, ejecuta el análisis y conserva los artefactos. La parte cualitativa valora si se ha comprendido la consulta, si las métricas son pertinentes, si la ejecución analítica es adecuada y si la interpretación final resulta fiel, clara, útil y prudente.

La batería de 50 casos distribuye las consultas en los niveles A, B y C, con una exigencia progresiva. Las dimensiones cualitativas reciben una preevaluación heurística que ayuda a localizar casos problemáticos. Sin embargo, estas reglas son aproximadas y no sustituyen la revisión humana. La combinación de ambos enfoques permite distinguir, por ejemplo, entre un caso que falla al ejecutar el código y otro que obtiene resultados válidos, pero los explica de manera insuficiente.

2.6. Comparativa y posicionamiento de la propuesta

La Tabla 2.1 resume las líneas revisadas y muestra cómo se relacionan con el sistema desarrollado. El objetivo concreto es integrar un modelo de lenguaje con generación controlada de código y análisis reproducible de datos históricos.

Tabla 2.1: Comparación entre los trabajos relacionados y la propuesta desarrollada.

Trabajo o línea	Aportación principal	Relación con este trabajo
FinBERT, BloombergGPT y FinGPT	Modelos entrenados o adaptados al lenguaje financiero [3], [4], [5].	Se centran en el modelo; la propuesta estudia la integración controlada de un modelo disponible.
FinRobot y TradingAgents	Agentes para investigación, valoración o trading [7], [8].	Inspiran la separación de responsabilidades, pero el sistema no toma decisiones de inversión ni actúa sobre el mercado.
ReAct, Toolformer y LangGraph	Herramientas externas y flujos coordinados mediante estados y transiciones [9], [10], [12].	Respaldan el uso de herramientas y un estado común; aquí el recorrido está fijado para favorecer el control.
Codex, DS-1000 y seguridad de código	Síntesis, evaluación y riesgos del código producido por modelos [13], [14], [15].	Justifican la generación de código bajo contrato y su validación antes de ejecutarlo.
Alucinaciones y RAG	Estudio de respuestas no fieles y anclaje en fuentes externas [16], [17].	La respuesta se apoya en los CSV locales y en los resultados obtenidos, aunque no se utiliza recuperación documental.
HELM y G-Eval	Evaluación multidimensional y calidad del texto generado [18], [19].	Fundamentan la combinación de indicadores técnicos, dimensiones cualitativas y revisión humana.

En conjunto, el trabajo ocupa una posición intermedia entre un asistente puramente conversacional y un agente financiero autónomo. Un único modelo planifica el análisis, genera el código y explica los resultados, mientras que las capas deterministas se encargan de la validación, la ejecución y el registro de artefactos. Esta separación permite estudiar el valor práctico del modelo sin atribuirle capacidades que no se han implementado ni evaluado.

Capítulo 3

Datos y material de trabajo

3.1. Alcance del capítulo

El sistema analiza series financieras históricas almacenadas en archivos locales. Trabajar con un conjunto de datos conocido y estable permite repetir una misma consulta en condiciones comparables, sin depender de la disponibilidad de un servicio externo ni de los cambios posteriores del mercado.

El material experimental se organiza en tres grupos. El primero reúne los CSV originales y la información asociada a cada descarga. El segundo contiene las tablas preparadas para estudiar la estructura y la calidad de los datos. El tercero corresponde a la batería de 50 consultas utilizada para evaluar el comportamiento del modelo. Este capítulo presenta estos recursos y explica en qué condiciones se emplean; el procedimiento de evaluación y sus resultados se desarrollan más adelante.

3.2. Procedencia y obtención de los datos

Las series históricas se obtuvieron mediante `yfinance` [20], una herramienta que facilita la descarga de parte de la información publicada en Yahoo Finance [21]. Los datos incluyen los campos habituales de mercado: precio de apertura, máximo, mínimo, cierre, cierre ajustado y volumen.

El conjunto incorpora acciones, fondos cotizados, un índice bursátil, una criptomoneda, una divisa y una materia prima. La selección es intencional y responde a criterios de variedad, no a un muestreo estadístico del mercado. No se pretende representar de forma exhaustiva todos los instrumentos financieros, sino reunir situaciones suficientemente distintas para comprobar si el flujo se adapta a uno o varios activos, a intervalos diarios y horarios, y a horizontes que van desde una semana hasta varios años.

Cada descarga se conserva como un archivo CSV acompañado de sus metadatos. Esta información incluye la consulta de origen, los símbolos financieros, las fechas o el

periodo solicitado y el intervalo de observación. De este modo, los valores numéricos permanecen vinculados a las condiciones en las que fueron obtenidos.

La descarga y el análisis se realizan en momentos distintos. Durante la ejecución de una consulta, el sistema no solicita información nueva a Yahoo Finance, sino que trabaja exclusivamente con los archivos locales seleccionados para ese caso. Esto reduce la variabilidad entre pruebas y permite revisar posteriormente qué información se utilizó en cada análisis.

3.3. Inventario y cobertura

El catálogo inicial contiene diez consultas de descarga, aunque varias comparten el mismo conjunto de datos. Las dos formulaciones relativas a AAPL, por ejemplo, utilizan un único CSV; lo mismo ocurre con las consultas sobre QQQ y SPY. Por este motivo, el conjunto fijo está formado por ocho archivos, de los que se obtienen diez series al separar los casos que incluyen dos símbolos.

La Tabla 3.1 resume la cobertura observada después de normalizar cada combinación de caso y activo. En total se obtienen 5.080 observaciones.

Conjunto	Activos	Intervalo	Rango observado	Obs.
NVDA, cinco años	NVDA	Diario	17/03/2021– 16/03/2026	1.255
NVDA frente a AMD	NVDA, AMD	Diario	18/03/2024– 16/03/2026	1.000
AAPL, tres meses	AAPL	Diario	17/12/2025– 16/03/2026	60
QQQ frente a SPY	QQQ, SPY	Diario	02/01/2024– 30/12/2024	502
S&P 500 desde 2020	^GSPC	Diario	02/01/2020– 16/03/2026	1.558
Bitcoin durante 2024	BTC-USD	Diario	01/01/2024– 30/12/2024	365
EUR/USD, diez días	EURUSD=X	Horario	04/03/2026– 17/03/2026	224
Oro, una semana	GC=F	Horario	10/03/2026– 17/03/2026	116

Tabla 3.1: Inventario y cobertura temporal de los datos utilizados.

La cobertura real no siempre coincide exactamente con las fechas solicitadas. Los mercados tradicionales no generan observaciones durante fines de semana y festivos, mientras que Bitcoin cotiza de manera continua. Cuando la petición utiliza un periodo relativo, como tres meses o cinco años, las fechas también dependen del momento en que se realizó la descarga. El análisis toma, por tanto, las fechas presentes en cada CSV y no presupone que todos los activos compartan el mismo calendario.

En las series horarias, las marcas temporales conservan la referencia UTC. Esta convención permite trabajar con una base temporal común, aunque no elimina las diferencias entre los horarios efectivos de negociación de cada mercado.

3.4. Estructura y normalización

Los archivos descargados mediante `yfinance` emplean columnas de tipo OHLCV. La Tabla 3.2 recoge su función dentro del conjunto de datos.

Campo	Contenido	Uso en el análisis
Open	Precio de apertura.	Contextualización de la sesión y cálculos que requieren el inicio del intervalo.
High	Precio máximo.	Estudio de máximos y amplitud del movimiento.
Low	Precio mínimo.	Estudio de mínimos y amplitud del movimiento.
Close	Precio de cierre.	Variable principal para rentabilidad, volatilidad, drawdown y medias móviles.
Adj Close	Cierre ajustado, cuando está disponible.	Se conserva como referencia, aunque Close es la variable principal del trabajo.
Volume	Volumen negociado o informado.	Medida complementaria, no obligatoria para todos los tipos de activo.

Tabla 3.2: Campos financieros presentes en los CSV.

Cuando un archivo contiene varios símbolos, las columnas presentan dos niveles: uno identifica el activo y otro el campo financiero. Para facilitar su análisis, esta estructura se transforma a un formato largo en el que cada fila corresponde a un activo y un instante concretos. La versión normalizada conserva también el caso de origen, el intervalo de observación y los campos OHLCV.

Como resultado de esta transformación se obtienen tres recursos: el conjunto normalizado, un resumen por archivo y otro por serie. Estos materiales facilitan la inspección de los datos y evitan que la evaluación dependa de una interpretación manual de las cabeceras originales.

Cada caso de análisis conserva la consulta, los símbolos financieros, las fechas o el periodo, el intervalo y los CSV que puede utilizar. Esta selección establece un límite explícito: el sistema debe trabajar únicamente con esos datos y no puede descargar información adicional durante la ejecución.

3.5. Análisis exploratorio y calidad

Antes de comenzar la evaluación se realizó un análisis exploratorio para comprobar la cobertura, la consistencia y la utilidad de los datos. En esta fase todavía no se valoran las respuestas del modelo. El propósito es detectar problemas básicos en el material de partida que, de otro modo, podrían confundirse con fallos del sistema.

Indicador	Resultado
Consultas en el catálogo de descarga	10
CSV históricos	8
Serie por activo y caso	10
Observaciones normalizadas	5.080
Rango temporal total	02/01/2020–17/03/2026
Valores ausentes en Close	0
Fechas duplicadas dentro de cada serie	0

Tabla 3.3: Resumen de cobertura y calidad del conjunto de datos.

La ausencia de valores nulos en el cierre es especialmente importante, ya que esta columna interviene en la mayor parte de las métricas solicitadas. Tampoco se detectaron fechas duplicadas dentro de una misma serie. Por tanto, las rentabilidades, la volatilidad y el drawdown pueden calcularse sin recurrir a métodos de imputación.

El precio de cierre se utiliza como referencia principal para mantener un criterio uniforme entre todos los activos. Sin embargo, en horizontes largos puede diferir del cierre ajustado por el efecto de dividendos, desdoblamientos u otras operaciones corporativas. Esta elección debe tenerse en cuenta al interpretar rentabilidades históricas, especialmente en acciones y fondos cotizados.

El volumen, en cambio, requiere una lectura más prudente. En la serie EUR/USD aparece con valor cero en todas las observaciones y también se registran algunos ceros en el S&P 500 y el oro. Esto no invalida las series de precios, pero impide tratar el volumen como una variable homogénea u obligatoria en todos los casos.

El análisis exploratorio también muestra comportamientos financieros diversos: series de crecimiento prolongado, periodos con caídas acusadas, activos de elevada volatilidad y comparaciones entre instrumentos con perfiles distintos. Esta variedad evita que la evaluación se limite a repetir una misma operación sobre datos muy similares.

Las medidas anualizadas calculadas para los intervalos horarios deben interpretarse como aproximaciones descriptivas. Su valor depende del factor de anualización adoptado y de las horas efectivas de negociación de cada activo. Por ello, se emplean para inspeccionar la coherencia del conjunto, no como una referencia financiera universal.

3.6. Batería de consultas

A partir de los ocho CSV se construyó una batería de 50 consultas. No se trata de cincuenta descargas diferentes, sino de distintas tareas planteadas sobre un conjunto estable de datos. Los casos combinan operaciones, métricas y formatos de salida diversos, lo que permite estudiar la adaptación del modelo sin que los cambios en la fuente de información alteren la comparación.

Cada caso del catálogo incluye un identificador, el nivel de dificultad, el objetivo, el conjunto de datos, las salidas esperadas y los parámetros necesarios para realizar el análisis. La distribución se muestra en la Tabla 3.4.

Las salidas esperadas describen los componentes que debería contener cada análisis, como una rentabilidad, una tabla o una explicación de las limitaciones. No proporcionan de antemano las cifras que debe devolver el sistema. Los valores numéricos se calculan durante la ejecución a partir del CSV correspondiente, lo que permite comprobar después si la respuesta final respeta los resultados obtenidos.

Nivel	Casos	Porcentaje	Características principales
A	10	20 %	Consultas directas que sirven como referencia básica.
B	20	40 %	Varias métricas, comparaciones y salidas tabulares o visuales.
C	20	40 %	Tareas multicriterio, restricciones simultáneas, formatos exigentes y análisis avanzados.
Total	50	100 %	Progresión desde operaciones básicas hasta peticiones de mayor exigencia.

Tabla 3.4: Distribución de la batería de 50 consultas por nivel.

La distribución concede más peso a los niveles B y C porque concentran las tareas que mejor permiten evaluar la capacidad de adaptación del modelo. En estos casos debe combinar varias métricas, relacionar resultados, respetar formatos y elaborar explicaciones ajustadas al usuario. Los diez ejemplos de nivel A se mantienen como referencia para comprobar que el sistema resuelve correctamente las operaciones básicas, pero se evita que las consultas sencillas dominen el resultado agregado. En conjunto, el 80 % de la batería corresponde a tareas intermedias o avanzadas. Este reparto responde al propósito experimental del trabajo y no pretende reproducir la frecuencia con la que cada tipo de consulta aparecería en un entorno real.

Los conjuntos no se utilizan con la misma frecuencia. Bitcoin aparece en ocho casos; QQQ y SPY, también en ocho; AAPL, NVDA a cinco años y NVDA frente a AMD, en siete cada uno; el S&P 500, en siete; la serie de oro, en cuatro; y EUR/USD, en tres. Uno de los casos avanzados combina los datos de Bitcoin y del S&P 500, por lo que el número de apariciones de los conjuntos supera en una unidad al número total

de consultas. Esta distribución amplía la variedad de tareas, pero también implica que algunos datos tienen más peso que otros en los resultados agregados.

No se establece una partición entre entrenamiento, validación y prueba porque estos datos no se utilizan para ajustar el modelo. Todo el material actúa como conjunto de evaluación del sistema completo. Por este motivo, los resultados describen el comportamiento del prototipo en los 50 casos definidos, pero no permiten inferir por sí solos su rendimiento ante cualquier consulta financiera posible.

3.7. Reproducibilidad y limitaciones

Conservar los CSV y sus metadatos permite reconstruir el material utilizado sin necesidad de descargarlo de nuevo. Las tablas procesadas pueden obtenerse nuevamente a partir de los datos originales, mientras que el catálogo de 50 casos conserva las condiciones empleadas durante la evaluación. Quedan así diferenciados los datos de partida, sus transformaciones y las consultas experimentales.

El conjunto presenta, no obstante, varias limitaciones. Se ha diseñado para validar un prototipo y su tamaño es reducido frente a la diversidad real de activos, mercados y condiciones financieras. Además, algunas descargas emplean periodos relativos, por lo que repetirlas en otra fecha produciría una cobertura diferente. Los metadatos conservan los parámetros de cada petición, pero no incluyen una marca temporal explícita de la descarga; incorporarla reforzaría la trazabilidad de futuras versiones. Por último, los datos proceden de una herramienta abierta no afiliada ni avalada por Yahoo y no deben considerarse un suministro garantizado para producción [20].

También quedan fuera del alcance los datos en tiempo real, los datos fundamentales de las empresas, las noticias, los indicadores macroeconómicos y los costes asociados a una operación financiera. Tampoco se homogeneizan monedas ni calendarios de negociación para realizar comparaciones directas entre mercados distintos. En consecuencia, las respuestas del sistema describen y analizan las series históricas disponibles, pero no deben interpretarse como predicciones ni como recomendaciones de inversión.

Capítulo 4

Metodología

4.1. Enfoque metodológico

La metodología combina el desarrollo incremental del prototipo con una evaluación empírica y descriptiva de su funcionamiento. El propósito no es entrenar un modelo financiero, comparar varios LLM ni medir su capacidad predictiva. El interés se centra en comprobar hasta qué punto un único modelo puede comprender una consulta financiera, adaptarse a sus requisitos, preparar el análisis necesario y redactar una respuesta clara a partir de resultados calculados sobre datos históricos.

Esta perspectiva obliga a evaluar algo más que el texto que recibe el usuario. Una respuesta puede estar bien redactada y, sin embargo, apoyarse en un cálculo incorrecto; del mismo modo, una ejecución técnicamente válida puede terminar en una explicación poco clara o alejada de lo solicitado. Por este motivo, se estudia el flujo completo y se conserva la información generada en cada una de sus etapas.

4.1.1. Preguntas de evaluación

A partir de los objetivos del trabajo, la evaluación se articula en torno a cuatro preguntas:

1. ¿Es capaz el modelo de interpretar la consulta y elaborar un plan coherente con los activos, el periodo, las métricas y el formato solicitados?
2. ¿Puede generar un análisis adaptado a la tarea, respetar las restricciones del sistema y producir una salida utilizable?
3. ¿Se mantiene la respuesta final fiel a los resultados obtenidos y los comunica de forma clara, útil y prudente?
4. ¿Se observan diferencias en la estabilidad del flujo o en la calidad de las respuestas al aumentar la dificultad de los casos A, B y C?

Estas preguntas no pueden responderse con una única medida. Las dos primeras requieren observar la planificación y la ejecución del análisis; la tercera exige revisar la relación entre los resultados calculados y el texto final; y la cuarta se estudia agrupando los resultados por nivel de dificultad.

4.2. Proceso de desarrollo

El desarrollo se organizó de forma incremental. En lugar de construir desde el principio un sistema con todas sus funciones, se añadieron y comprobaron los componentes de manera progresiva:

1. Se definió una entrada estructurada que reúne la consulta, los activos, el periodo solicitado, el intervalo temporal y los datos disponibles.
2. Se incorporaron comprobaciones iniciales para evitar que una petición incompleta o incompatible con los datos disponibles alcanzara las etapas posteriores.
3. Se separó la intervención del LLM en tres tareas: planificación del análisis, generación del código e interpretación de los resultados.
4. Se establecieron formatos estructurados para comunicar las etapas del flujo y reducir la ambigüedad de las respuestas del modelo.
5. Se incorporaron controles sobre el análisis generado antes de permitir su ejecución.
6. Se aisló la fase de cálculo en un proceso independiente, con un tiempo máximo y registro de sus resultados y errores.
7. Se añadieron ciclos acotados de reparación para corregir los análisis rechazados o fallidos a partir del error observado.
8. Se conservaron las trazas necesarias para revisar cada caso sin depender únicamente de la respuesta final.
9. Se construyó una batería de 50 consultas para evaluar el sistema con distintos niveles de dificultad y requisitos de presentación.

Este proceso permitió comprobar cada ampliación antes de aumentar la complejidad del prototipo. También ayudó a mantener separados tres planos con funciones diferentes: los datos determinan qué información está disponible, la ejecución produce los cálculos y la interpretación convierte esos resultados en una explicación comprensible. Esta separación evita atribuir a la fase de redacción operaciones que deben quedar resueltas y registradas previamente.

4.2.1. Verificación previa del prototipo

Antes de ejecutar la batería se realizaron pruebas automatizadas sobre los componentes que no dependen de una respuesta real del modelo. Para ello se simulaban sus tres intervenciones y se comprobó que diferentes tipos de consulta recorrieran correctamente las fases de planificación, generación, ejecución e interpretación. Esta verificación permite distinguir un fallo del sistema de una respuesta variable del modelo.

También se comprobaron situaciones de error previsibles, como una consulta vacía, una fecha con formato incorrecto, la ausencia de datos o la imposibilidad de acceder al modelo. En estos casos, el flujo debe finalizar de forma controlada y ofrecer una explicación comprensible, en lugar de continuar con información incompleta.

Las pruebas también cubren la configuración del modelo, el tratamiento correcto de caracteres y la detección de respuestas que no respetan el formato esperado. No evalúan la calidad financiera del contenido, pero proporcionan una base estable antes de iniciar la evaluación empírica.

4.3. Diseño de la evaluación

4.3.1. Unidad de análisis

Cada ejemplo de la batería se trata como un caso independiente. El sistema no conserva una conversación previa ni utiliza la respuesta de otro ejemplo como contexto. Todos los casos recorren, por tanto, la misma secuencia general:

entrada, planificación, generación del análisis, validación, ejecución e interpretación.

La evaluación se realiza sobre el resultado del flujo completo. Al conservarse los productos intermedios, también es posible comprobar si el modelo entendió la tarea, si seleccionó métricas razonables, si el análisis respetó las condiciones establecidas y si la explicación final se mantuvo fiel a los resultados obtenidos.

4.3.2. Batería de consultas

La batería está formada por 50 ejemplos contruidos sobre los datos locales descritos en el capítulo anterior. Como recoge la Tabla 3.4, contiene 10 casos de nivel A, 20 de nivel B y 20 de nivel C. El reparto no busca reproducir la frecuencia real de cada tipo de consulta, sino concentrar la evaluación en las situaciones que plantean mayores dificultades al modelo.

Los diez casos de nivel A proporcionan una referencia básica: permiten comprobar que el sistema interpreta peticiones directas y resuelve operaciones sencillas antes de valorar tareas más complejas. Los niveles B y C reúnen el 80 % de la batería porque

exigen combinar métricas, relacionar resultados, trabajar con comparaciones o series temporales, respetar formatos y adaptar la explicación a distintos requisitos. El nivel C añade, además, restricciones simultáneas y análisis avanzados, como el estudio del riesgo de cola, los episodios de caída y recuperación, la alineación de activos con calendarios diferentes o la delimitación precisa del instrumento analizado.

Esta distribución introduce deliberadamente más presión en los niveles donde pueden aparecer errores de planificación, cálculo o redacción. Por tanto, los resultados globales deben interpretarse teniendo en cuenta que representan una evaluación orientada a consultas intermedias y avanzadas, y no una muestra equilibrada del conjunto de peticiones que podría formular un usuario.

Los niveles se aplican con el siguiente criterio:

- **Nivel A:** consultas directas, centradas en una o pocas métricas y con una presentación sencilla.
- **Nivel B:** consultas que combinan varias métricas, comparaciones y requisitos de salida más detallados.
- **Nivel C:** consultas con varios criterios, restricciones simultáneas o una estructura de presentación más exigente.

La clasificación A, B o C incluida en el catálogo representa el nivel esperado de cada caso. Sin embargo, este valor no se impone al planificador. El modelo recibe la consulta y debe determinar por sí mismo la complejidad del análisis, por lo que posteriormente puede compararse el nivel previsto en la batería con el asignado durante la planificación.

La dificultad no depende únicamente del número de activos o de métricas. También intervienen la necesidad de relacionar resultados, respetar un formato concreto, preparar información tabular o visual y expresar limitaciones de forma adecuada. De este modo, los niveles reflejan el esfuerzo conjunto de comprensión, cálculo y comunicación que exige cada consulta.

4.4. Protocolo de ejecución

Antes de iniciar un caso se recuperan del catálogo la consulta, los activos, las fechas, el intervalo y los datos necesarios. La entrada se valida antes de invocar el modelo. Si falta información imprescindible o alguno de los datos no está disponible, el flujo finaliza con un mensaje de error controlado.

Los tres agentes utilizan el mismo modelo, `openai/gpt-oss-20b`, a través de un servicio de inferencia compatible. La configuración establece una temperatura de 0,1, un máximo de 4096 tokens por respuesta y un tiempo de espera de 120 segundos para

cada petición. La baja temperatura busca reducir variaciones innecesarias, aunque no elimina por completo el carácter no determinista de la generación.

Una ejecución ordinaria requiere al menos tres intervenciones del LLM: una para elaborar el plan, otra para generar el análisis y una tercera para interpretar los resultados. La planificación, la generación y la interpretación pueden repetirse hasta tres veces cuando la respuesta no respeta el formato esperado. Además, si el análisis no supera la validación o falla durante su ejecución, se contempla un intento de reparación a partir del error detectado. Por ello, el número de llamadas no es necesariamente idéntico en todos los casos.

El análisis aceptado se ejecuta en un proceso independiente con un límite de 60 segundos y debe producir un resultado estructurado y procesable. A continuación, el agente de interpretación recibe el plan y los resultados, pero no accede directamente a los datos originales ni vuelve a calcular las métricas. Su función consiste en explicar los valores obtenidos, no en sustituir la fase de análisis.

La batería puede ejecutarse completa en una sola sesión. Sin embargo, para reducir el impacto de posibles interrupciones, problemas de disponibilidad o tiempos de espera, también puede dividirse en bloques de 5 o 10 casos. Los resultados se conservan de manera incremental, de modo que una ejecución interrumpida puede reanudarse sin repetir los ejemplos ya registrados. Esta estrategia no modifica el tratamiento individual de los casos, pero hace más robusto el proceso de evaluación.

4.5. Cadena de validación

La validación se plantea como una sucesión de controles. Cada uno responde a un riesgo distinto y evita que la evaluación dependa exclusivamente de que el texto final resulte convincente.

Etapas	Comprobación realizada	Finalidad
Entrada	Consulta, activos, fechas, intervalo y datos requeridos.	Evitar peticiones incompletas o incompatibles con los datos.
Planificación	Nivel A/B/C, objetivo, métricas, columnas y salidas previstas.	Comprobar que el análisis responde a la petición.
Generación	Correspondencia con el plan y producción de una salida estructurada.	Mantener la coherencia entre la planificación y el análisis.
Seguridad	Operaciones, dependencias y accesos utilizados.	Rechazar acciones no permitidas.
Ejecución	Estado final, resultados, errores y duración.	Confirmar que el análisis puede realizarse correctamente.
Interpretación	Fidelidad, claridad, limitaciones y prudencia financiera.	Evitar afirmaciones no respaldadas o recomendaciones.
Trazabilidad	Entrada, plan, análisis, resultados, avisos y respuesta final.	Facilitar la revisión y reproducción de cada caso.

Tabla 4.1: Cadena de validación aplicada a cada ejecución.

La superación de una etapa no garantiza por sí sola la calidad global. Un análisis puede ejecutarse correctamente y calcular métricas poco pertinentes, o una interpretación puede ser clara pero omitir una limitación importante. Por esta razón, los controles técnicos se complementan con una valoración de la calidad analítica y comunicativa.

4.6. Métricas de evaluación

4.6.1. Indicadores técnicos

Para cada caso se registran indicadores que pueden obtenerse directamente del estado del flujo:

- estado final de la ejecución;
- existencia de un plan analítico estructurado;
- generación y conservación del análisis;
- presencia de una salida ejecutada y procesable;
- existencia de una respuesta final;
- inclusión de datos tabulares o visuales cuando la consulta los requiere;
- ausencia de expresiones que puedan interpretarse como recomendaciones de compra o venta;

- avisos que permiten identificar regeneraciones o reparaciones, junto con los errores encontrados;
- duración total de la ejecución.

Estos indicadores permiten medir la completitud y la estabilidad del sistema. También ayudan a diferenciar los casos resueltos de aquellos en los que el flujo finaliza de forma controlada después de detectar un problema.

Un caso se considera completado cuando alcanza la interpretación final y dispone de un plan, un análisis ejecutado y una respuesta procesable. Los errores controlados se registran por separado: demuestran que el sistema ha detenido el flujo de forma segura, pero no se contabilizan como casos resueltos.

4.6.2. Calidad de la respuesta

La corrección técnica no describe por completo la utilidad de una respuesta. Para incorporar esta dimensión se definieron cinco criterios, puntuados de 0 a 2:

1. **Comprensión de la consulta:** valora si se identifican correctamente los activos, el objetivo, el periodo y el nivel de complejidad.
2. **Pertinencia de las métricas:** comprueba si las medidas seleccionadas resultan adecuadas para responder a lo solicitado.
3. **Calidad de la ejecución analítica:** considera si el flujo produce una salida estructurada, completa y coherente con el plan.
4. **Fidelidad de la interpretación:** revisa si la explicación se apoya en los resultados obtenidos y evita añadir conclusiones no justificadas.
5. **Claridad, utilidad y prudencia:** valora la organización del texto, su capacidad explicativa y la ausencia de recomendaciones financieras improcedentes.

La suma de los cinco criterios produce una puntuación máxima de 10. Como primera aproximación se calcula una preevaluación heurística mediante reglas sencillas. La comprensión se estima a partir de la presencia de los activos y la coincidencia del nivel; la pertinencia, mediante términos asociados a las salidas esperadas; y la ejecución, mediante el estado del flujo y la existencia de resultados. La fidelidad y la claridad se aproximan a partir de la completitud, la ausencia de expresiones prohibidas y una longitud mínima de la respuesta.

Esta puntuación sirve para ordenar la revisión y detectar casos problemáticos, pero no demuestra por sí sola que las métricas estén bien calculadas ni que cada afirmación reproduzca con exactitud la salida ejecutada. Tampoco debe interpretarse como

una valoración humana definitiva. Cualidades como la precisión de una explicación, la suficiencia del análisis o la utilidad real de una comparación requieren revisar las evidencias del caso.

4.6.3. Protocolo de revisión humana

La revisión humana se aplica como contraste sobre casos representativos de los niveles A, B y C. Se apoya en la misma escala de cinco dimensiones y utiliza como evidencia la consulta original, el plan, los resultados obtenidos y la respuesta final. Su propósito no es repetir las comprobaciones automáticas, sino valorar aspectos que dependen del contexto: si el análisis responde realmente a la intención de la consulta, si las métricas permiten sostener las conclusiones y si la redacción comunica los resultados sin exagerarlos.

Para reducir la arbitrariedad, la rúbrica define qué significa obtener 0, 1 o 2 puntos en cada criterio. Aun así, la valoración conserva un componente subjetivo, especialmente cuando la realiza una sola persona. Por ello, las revisiones manuales se utilizan como complemento y deben acompañarse de observaciones y evidencias concretas, en lugar de presentarse como una medida exacta de la calidad.

4.7. Tratamiento de los resultados

El análisis de la batería tiene un carácter descriptivo. Los resultados se agrupan de forma global y por nivel A, B y C. Se consideran, entre otros aspectos, la proporción de ejecuciones completadas, la frecuencia de errores controlados, los tiempos de respuesta, la necesidad de reparaciones y las puntuaciones de calidad.

La comparación entre niveles permite observar si el aumento de la complejidad afecta a la estabilidad del flujo o a la calidad de la respuesta. También se revisan los fallos de forma cualitativa para identificar patrones, ya que dos casos con el mismo estado final pueden haber fallado por motivos diferentes. Los valores obtenidos y su interpretación se presentan en el capítulo dedicado a los resultados, mientras que este capítulo se limita a definir el procedimiento utilizado para obtenerlos.

No se aplican pruebas de significación estadística ni se pretende generalizar el comportamiento observado a cualquier consulta financiera. La batería representa un conjunto concreto de tareas diseñadas para este prototipo y ejecutadas sobre datos históricos locales.

4.8. Reproducibilidad y limitaciones metodológicas

La reproducibilidad se apoya en el uso de un catálogo fijo, datos locales, una configuración conocida y el almacenamiento de los productos de cada ejecución. Estos elementos permiten volver a lanzar un caso y revisar qué información recibió y produjo cada etapa.

No obstante, existen varias limitaciones. Se evalúa un único modelo y se realiza una ejecución principal por caso, por lo que no puede medirse de forma sólida la variabilidad entre repeticiones. Tampoco se establece una semilla fija para la generación, y una temperatura baja no garantiza respuestas idénticas. La disponibilidad, la latencia y la carga del servicio pueden influir en los tiempos y en algunos errores.

Otra limitación se encuentra en la obtención de los datos. El prototipo no descarga automáticamente una serie financiera a partir de la consulta del usuario, sino que trabaja con datos seleccionados y almacenados previamente. Esta decisión reduce su autonomía, pero evita que una expresión ambigua conduzca al análisis de un instrumento distinto del que se pretendía estudiar. Por ejemplo, una petición sobre el “oro” puede referirse al precio de referencia de la LBMA, a los futuros negociados en COMEX o a un producto cotizado como SPDR Gold Shares. Aunque todos proporcionan exposición al mismo activo subyacente, no representan exactamente la misma serie ni comparten necesariamente horarios, condiciones de negociación o forma de cálculo [22], [23], [24].

Por este motivo, la selección y descarga manual permite comprobar antes de la ejecución qué instrumento, símbolo, mercado, moneda, intervalo y fuente corresponden a la consulta. Esta preparación mejora el control sobre los datos y facilita la reproducción de los experimentos, aunque exige disponer de las series con antelación. Como ampliación futura, el sistema podría incorporar una fase de identificación del activo que consultara una fuente externa, presentara al usuario las alternativas encontradas y solicitara confirmación antes de descargar los datos. De este modo se aumentaría la autonomía sin asumir de forma silenciosa qué instrumento quiso mencionar el usuario.

La distribución de la batería tampoco es uniforme: los niveles B y C tienen más peso que el nivel A, y varios casos reutilizan los mismos conjuntos de datos con objetivos diferentes. Este desequilibrio es deliberado, ya que el propósito principal es examinar el comportamiento del modelo ante tareas que exigen combinar cálculos, estructuras de salida y requisitos de redacción. Como contrapartida, los resultados globales describen mejor el rendimiento ante consultas intermedias y avanzadas que ante el conjunto general de posibles peticiones financieras. Además, el catálogo define los componentes esperados de cada salida, pero no incorpora una respuesta numérica de referencia para todos los casos. En consecuencia, completar el flujo y producir un resultado estructurado no basta para demostrar la corrección financiera de cada cálculo; esta debe

contrastarse mediante los resultados intermedios y la revisión del caso.

La preevaluación automática de la calidad utiliza reglas aproximadas y la revisión manual puede estar condicionada por el criterio de la persona evaluadora. Al no intervenir varios revisores independientes, tampoco puede calcularse el grado de acuerdo entre evaluadores.

Por último, los controles previos y la ejecución en un proceso independiente reducen el riesgo asociado al análisis generado, pero no equivalen a un aislamiento completo del entorno. Esta cuestión se aborda con mayor detalle en el capítulo dedicado a la seguridad y al uso responsable.

Capítulo 5

Diseño del sistema

Este capítulo presenta el diseño del sistema desde una perspectiva funcional. El objetivo no es describir todavía los módulos concretos de la implementación, sino explicar cómo se organiza el flujo, qué responsabilidad asume cada etapa y por qué se han separado la planificación, el cálculo y la redacción de la respuesta.

La Figura 5.1 actúa como hilo conductor del capítulo. Se trata de una representación resumida del recorrido completo: comienza con la petición del usuario, incorpora las validaciones y los tres agentes, muestra la generación y ejecución del análisis y termina con una respuesta final o con un error controlado. El diagrama completo utilizado durante el desarrollo contiene un mayor número de estados y detalles operativos, pero la versión resumida permite comprender las decisiones principales sin perder la lógica del sistema.

5.1. Objetivos y principios de diseño

El principal reto consiste en mantener la flexibilidad del lenguaje natural sin permitir que el modelo responda directamente con cifras que no hayan sido calculadas. Un LLM puede interpretar peticiones muy distintas y adaptar la explicación al usuario, pero no debe actuar como fuente de datos ni como sustituto de una ejecución verificable. A partir de esta idea se establecieron cinco principios.

1. **Separación de responsabilidades.** Comprender la consulta, realizar los cálculos y redactar la respuesta son tareas diferentes. Cada una se asigna a una etapa específica para evitar que una única generación mezcle decisiones analíticas, operaciones numéricas y conclusiones.
2. **Cálculos verificables.** Las cifras que llegan a la respuesta final deben proceder de una ejecución sobre los datos disponibles. El modelo propone el análisis y explica sus resultados, pero no inventa valores ni calcula de forma implícita en el texto.

3. **Control antes de la ejecución.** Todo procedimiento generado se revisa antes de ejecutarse. Esta capa intermedia permite rechazar resultados que no respetan el formato o las restricciones del sistema.
4. **Trazabilidad.** El flujo conserva la información necesaria para reconstruir cada caso y localizar el origen de un fallo. La respuesta final no es el único producto de la ejecución.
5. **Finalización controlada.** Cuando una etapa no puede completarse, el sistema detiene el recorrido y devuelve un mensaje comprensible. No se intenta ocultar el problema mediante una respuesta aparentemente válida.

Estos principios explican por qué la arquitectura no se limita a conectar una consulta con un modelo de lenguaje. Entre la entrada y la respuesta se introducen contratos estructurados, controles deterministas y una ejecución separada que reducen la dependencia del texto libre generado por el LLM.

5.2. Arquitectura general

El recorrido de la Figura 5.1 comienza cuando el usuario solicita un análisis financiero. La consulta se acompaña de una entrada estructurada con los activos, el periodo o las fechas, el intervalo temporal y los datos locales que pueden utilizarse. Esta combinación es importante: el lenguaje natural expresa la intención del usuario, mientras que la información estructurada delimita el alcance real del análisis.

La primera decisión del flujo es determinista. Antes de invocar al modelo se comprueba que la petición contiene la información mínima y que los datos necesarios están disponibles. Una entrada válida continúa hacia el agente analista; una entrada incompleta o incompatible termina con un mensaje de error. De este modo, el modelo no tiene que compensar por su cuenta la ausencia de información esencial.

El agente analista transforma la petición en un plan. A continuación, el agente generador convierte ese plan en un procedimiento de cálculo adaptado al caso. Entre esta generación y la ejecución se sitúa una capa de validación. Si el análisis cumple las condiciones establecidas, puede ejecutarse; si no las cumple, se permite una reparación acotada y una nueva comprobación. El procedimiento aceptado se ejecuta sobre los datos locales y produce resultados estructurados.

Por último, el agente interpretador recibe el plan y los resultados obtenidos. Su función es explicar, no recalcular. Si la ejecución ha sido correcta, redacta una respuesta final clara y prudente. Si alguna etapa crítica falla y no puede recuperarse, el sistema termina mediante una ruta de error controlado.

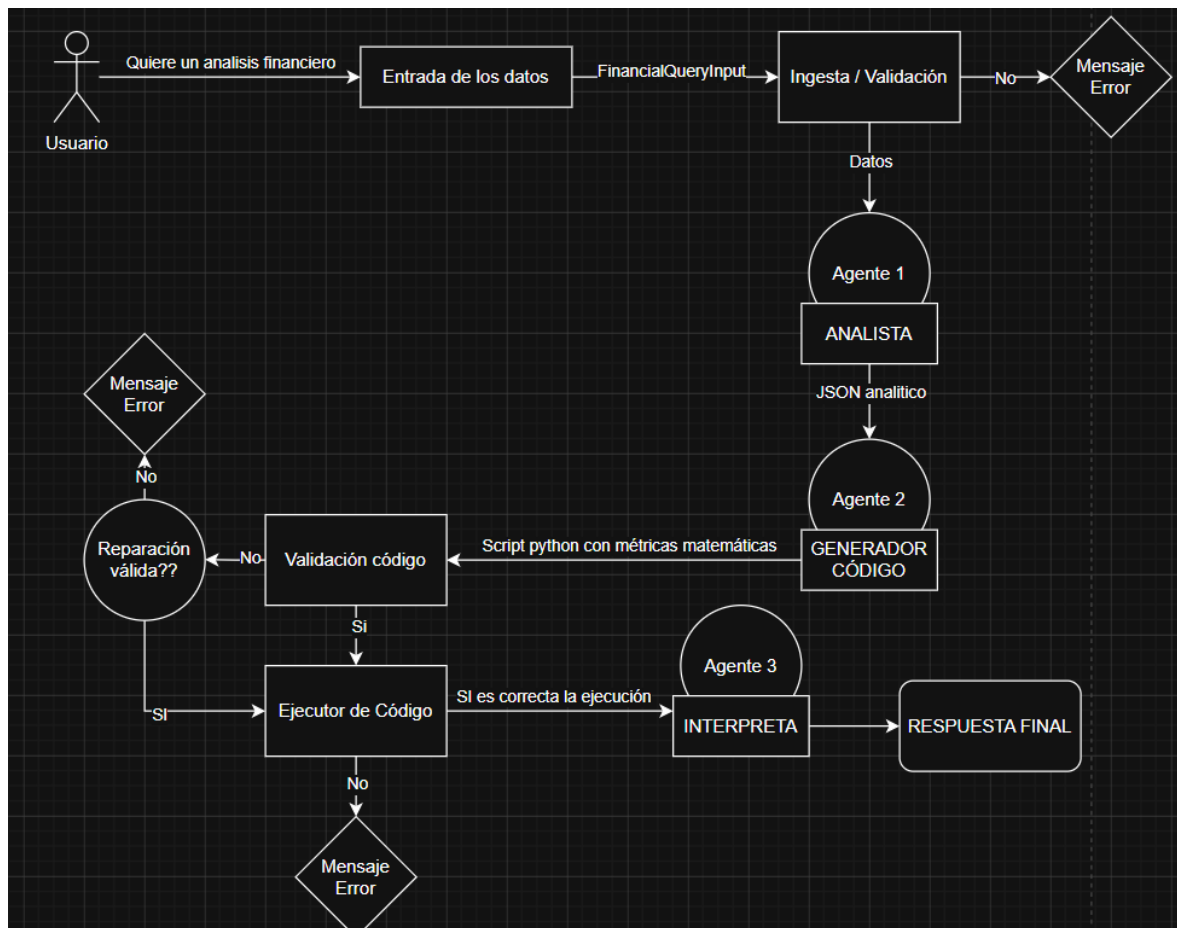


Figura 5.1: Flujo resumido del sistema desde la consulta del usuario hasta la respuesta final.

Bloque	Responsabilidad	Resultado
Entrada y validación	Delimitar la consulta y comprobar que puede procesarse con los datos disponibles.	Petición válida o explicación del problema.
Agente analista	Comprender la intención y definir el análisis necesario.	Plan analítico estructurado.
Agente generador	Preparar el procedimiento que realizará los cálculos solicitados.	Análisis ejecutable ajustado al plan.
Validación y ejecución	Revisar, ejecutar y registrar el procedimiento sobre datos locales.	Métricas, tablas, series, avisos o errores.
Agente interpretador	Convertir los resultados calculados en una explicación adaptada al usuario.	Respuesta final clara, fiel y prudente.

Tabla 5.1: Responsabilidades principales de la arquitectura.

5.3. Entrada y estado compartido

5.3.1. Entrada estructurada

La consulta del usuario no circula sola por el sistema. Se acompaña de los activos seleccionados, las fechas o el periodo de interés, el intervalo temporal, la localización de los datos y los avisos conocidos. Esta entrada estructurada cumple dos funciones. Por un lado, evita que cada agente tenga que volver a deducir información ya disponible. Por otro, establece un límite explícito: el análisis debe realizarse con esos datos y no con información externa añadida durante la generación.

El diseño admite tanto un periodo relativo como un intervalo definido mediante fechas. También permite uno o varios activos y uno o varios conjuntos de datos, como ocurre en las comparaciones que requieren alinear series con calendarios diferentes. La entrada no determina las métricas que deben calcularse; esa decisión corresponde al agente analista a partir de la intención expresada por el usuario.

Los datos se incorporan al flujo después de haber sido seleccionados y descargados. Como se explicó en el capítulo anterior, esta decisión reduce la autonomía del prototipo, pero permite conocer con precisión qué instrumento y qué serie se están utilizando. El sistema no intenta resolver silenciosamente ambigüedades como la diferencia entre un índice, un futuro o un producto cotizado relacionado con el mismo activo.

5.3.2. Estado del flujo

Durante la ejecución se mantiene un estado compartido que actúa como memoria del caso. Este estado comienza con la entrada original y se amplía a medida que avanza el recorrido. Puede contener el plan analítico, el procedimiento generado, los resultados de la ejecución, los avisos, los errores y la respuesta final.

El estado no debe confundirse con una conversación acumulada entre ejemplos. Cada consulta de la batería comienza de forma independiente y no utiliza como contexto las respuestas anteriores. Su finalidad es conservar la evolución de un único caso y permitir que cada etapa reciba únicamente la información necesaria.

Los cambios de estado permiten saber hasta dónde ha llegado una ejecución. Un caso puede encontrarse validado, planificado, preparado para ejecutar, ejecutado o completado. También puede finalizar por una entrada inválida, un análisis rechazado o un error durante el cálculo. Esta distinción evita reducir todo el comportamiento del sistema a una clasificación binaria entre éxito y fracaso.

5.4. Etapas del flujo

5.4.1. Ingesta y validación

La ingesta convierte la entrada recibida en el punto de partida común del flujo. En esta etapa se comprueba que la consulta no esté vacía, que se hayan indicado los activos, que las fechas tengan un formato válido y que los datos solicitados existan. También se conservan los avisos que puedan ser relevantes para las etapas posteriores.

Estas comprobaciones se realizan antes de cualquier intervención generativa. La razón es sencilla: un modelo puede producir una respuesta plausible incluso cuando faltan datos, pero esa respuesta no sería trazable. Si la petición no puede atenderse con la información disponible, el sistema lo comunica y no continúa hacia la planificación.

5.4.2. Agente 1: planificación analítica

El primer agente interpreta qué quiere obtener el usuario. Su salida no contiene resultados numéricos, sino una descripción estructurada del trabajo que debe realizarse. El plan recoge el objetivo analítico, el tipo y nivel de análisis, las métricas necesarias, las columnas que se utilizarán y los requisitos de presentación.

La planificación cumple una función de intermediación entre el lenguaje cotidiano y el cálculo. Consultas aparentemente parecidas pueden requerir procedimientos diferentes. Preguntar cuánto ha crecido un activo exige una rentabilidad acumulada; estudiar su riesgo puede requerir volatilidad y drawdown; y preparar un informe comparativo obliga además a alinear series, organizar tablas y expresar limitaciones.

El nivel A, B o C se decide en esta etapa a partir de la consulta. No se utiliza para alterar los datos, sino para ajustar la profundidad del análisis y el grado de estructura esperado en la salida. El plan queda registrado, lo que permite comprobar posteriormente si la generación y la respuesta final respetaron la intención inicial.

5.4.3. Agente 2: generación del análisis

El segundo agente recibe la entrada validada y el plan. Su responsabilidad es convertir ambos elementos en un procedimiento capaz de obtener las métricas y estructuras solicitadas. Esta separación evita pedir al mismo agente que decida qué debe hacerse y, al mismo tiempo, cómo debe ejecutarse.

La generación está sujeta a un contrato. El procedimiento debe trabajar únicamente con los datos proporcionados, utilizar las operaciones permitidas y devolver un resultado estructurado. Cuando la consulta requiere una tabla o una serie temporal, estas deben formar parte de la salida de manera procesable. La explicación dirigida al usuario queda fuera de esta etapa.

El modelo conserva flexibilidad para adaptar el cálculo a consultas diferentes, pero no controla el resto del entorno. La arquitectura limita sus entradas, revisa su salida y ejecuta el análisis mediante una etapa independiente.

5.4.4. Validación y reparación

Antes de ejecutar el análisis se comprueba que tenga una estructura válida y que no incorpore operaciones ajenas al alcance del sistema. Esta revisión actúa como frontera entre una propuesta generada y una acción que realmente puede realizarse.

Si el análisis no supera la validación, el sistema puede solicitar una corrección utilizando el plan, la versión rechazada y el motivo del rechazo. La reparación es limitada y no constituye una vía alternativa para evitar los controles. La nueva propuesta vuelve a someterse a la misma validación. Si tampoco cumple las condiciones, el caso termina de forma controlada.

Este mecanismo mejora la robustez ante errores corregibles, como un formato incompleto o una operación no admitida, sin convertir el flujo en un ciclo indefinido. El objetivo no es forzar que todos los casos terminen correctamente, sino permitir una recuperación razonable y conservar evidencia de que fue necesaria.

5.4.5. Ejecución controlada

Una vez validado, el procedimiento se ejecuta en un proceso independiente y con un tiempo máximo. La ejecución recibe la consulta, los activos, los datos disponibles y

el plan analítico. Como resultado produce las métricas, tablas o series necesarias para responder al usuario.

La separación entre generación y ejecución es una de las decisiones centrales del diseño. El modelo propone el análisis, pero las cifras aparecen únicamente después de aplicar ese procedimiento sobre los datos. Así se puede distinguir entre una explicación convincente y un resultado respaldado por una operación efectivamente realizada.

Durante esta etapa se registran la salida, los errores, el código de finalización, la duración y los productos necesarios para revisar el caso. Si el cálculo falla, se puede intentar una reparación a partir del error observado. La versión corregida vuelve a validarse antes de ejecutarse. Si el segundo intento tampoco funciona, el sistema no continúa hacia una interpretación ordinaria.

5.4.6. Agente 3: interpretación

El tercer agente recibe el plan y los resultados calculados. No accede de nuevo a los datos originales ni debe completar cifras ausentes. Su tarea consiste en seleccionar la información relevante, explicar las métricas y construir una respuesta proporcionada al nivel de la consulta.

Esta restricción reduce el riesgo de que la fase de redacción modifique el contenido analítico. La respuesta puede reorganizar y contextualizar los resultados, pero debe mantenerse fiel a ellos. También debe expresar las limitaciones del caso y evitar recomendaciones de compra o venta.

Cuando la ejecución produce listas o series extensas, el agente recibe una versión compactada que conserva los elementos necesarios para interpretar el resultado. La salida completa se mantiene almacenada como evidencia. De este modo se evita saturar el contexto del modelo sin perder la trazabilidad del análisis.

5.5. Rutas de error y recuperación

La Figura 5.1 muestra que el recorrido no tiene una única salida. La respuesta final es el resultado esperado, pero cada etapa crítica dispone de una ruta de finalización controlada. Esta organización permite informar del problema sin atribuirlo a una causa equivocada.

Origen	Tratamiento	Consecuencia
Entrada incompleta	Se detiene el flujo antes de invocar al modelo.	Mensaje con la información ausente o inválida.
Fallo de planificación	Se conserva el error de comunicación o de formato.	El análisis no llega a generarse.
Análisis rechazado	Se permite una reparación y se repite la validación.	Continuación o finalización controlada.
Error de ejecución	Se utiliza el error observado para un intento de corrección.	Nuevo cálculo validado o mensaje de error.
Fallo de interpretación	Se conservan los resultados ya calculados.	Respuesta de error sin perder la evidencia analítica.

Tabla 5.2: Principales rutas de error y recuperación del sistema.

Separar los errores por origen resulta especialmente importante para la evaluación. Una entrada inválida no indica una mala capacidad de cálculo; un análisis rechazado no equivale a una interpretación incorrecta; y una respuesta final fallida no elimina necesariamente los resultados obtenidos. El estado conservado permite distinguir estas situaciones.

La recuperación se mantiene deliberadamente acotada. Un número ilimitado de intentos aumentaría el coste, dificultaría la comparación entre casos y podría ocultar que el modelo no fue capaz de cumplir el contrato inicial. Por esta razón, la reparación se considera una ayuda puntual y también una señal que debe registrarse en los resultados.

5.6. Trazabilidad y conservación de evidencias

La trazabilidad atraviesa toda la arquitectura. Para cada caso se conserva información suficiente para reconstruir el recorrido desde la consulta hasta la respuesta. Esta propiedad permite revisar no solo qué respondió el sistema, sino también por qué llegó a esa respuesta.

Evidencia	Utilidad
Entrada original	Permite comprobar qué pidió el usuario y con qué datos podía responderse.
Plan analítico	Muestra cómo se interpretó la consulta y qué métricas se consideraron necesarias.
Análisis generado	Permite revisar el procedimiento utilizado para obtener los resultados.
Resultado ejecutado	Contiene las cifras, tablas, series y limitaciones producidas durante el cálculo.
Avisos y errores	Identifican reparaciones, rechazos y problemas encontrados durante el recorrido.
Respuesta final	Permite contrastar la explicación con el plan y los resultados obtenidos.

Tabla 5.3: Información conservada para garantizar la trazabilidad de cada caso.

Esta conservación también hace posible la evaluación por etapas. Si una respuesta es incompleta, puede revisarse si el problema comenzó en la planificación, apareció durante el cálculo o se introdujo al redactar. Del mismo modo, dos respuestas finales parecidas pueden proceder de recorridos muy distintos, por ejemplo, si una necesitó reparación y la otra se completó en el primer intento.

La batería de 50 consultas se apoya en esta propiedad. Las métricas técnicas se obtienen del estado del flujo y la revisión cualitativa utiliza la consulta, el plan, los resultados y la respuesta final. Sin esta información intermedia, la evaluación quedaría reducida a juzgar únicamente la apariencia del texto.

5.7. Decisiones de diseño y alcance

La arquitectura responde a decisiones concretas y también implica renunciaciones. Una de las principales es utilizar tres agentes especializados en lugar de pedir al modelo que resuelva toda la consulta en una única llamada. Esta segunda opción habría simplificado el recorrido y reducido el número de interacciones, pero también habría reunido en una misma respuesta la comprensión de la petición, la preparación de los cálculos y la redacción final. Ante un resultado incorrecto, sería difícil determinar si el modelo interpretó mal la consulta, eligió métricas inadecuadas, generó un cálculo erróneo o explicó de forma deficiente unos resultados correctos.

La división adoptada deja un producto revisable después de cada intervención: un plan analítico, un código ejecutable y una interpretación final. Cada uno puede contrastarse con el anterior antes de continuar. El plan permite comprobar qué entendió

el sistema; el código muestra cómo trató de llevarlo a la práctica; y la respuesta final puede compararse con los resultados realmente calculados. De este modo, la trazabilidad no se limita a conservar la contestación recibida por el usuario, sino que permite reconstruir las decisiones que condujeron hasta ella.

Esta separación también facilita la recuperación ante fallos. Si el problema aparece en el código, puede intentarse su reparación sin volver a interpretar la consulta ni rehacer una respuesta completa. Si el cálculo es correcto pero la explicación resulta insuficiente, el error queda situado en la última etapa. El coste es una mayor latencia y un mayor número de llamadas al modelo, pero se acepta porque el propósito del trabajo no es solo obtener una respuesta, sino poder examinar cómo se construye y evaluar por separado la comprensión, la generación de código y la capacidad de redacción.

La segunda decisión es intercalar etapas deterministas entre las intervenciones generativas. La validación de entrada, la revisión previa y la ejecución no dependen de que el modelo redacte una respuesta convincente. Estas etapas reducen la ambigüedad y establecen puntos de control claros.

La tercera es trabajar con datos locales previamente identificados. Esta opción limita la autonomía, pero evita seleccionar automáticamente un instrumento equivocado y mantiene estable la información utilizada en la evaluación. La posibilidad de buscar y confirmar activos antes de descargar sus datos se plantea como una ampliación futura, no como una responsabilidad del flujo actual.

La cuarta decisión es separar el resultado completo de la información enviada al agente interpretador. Las series extensas se conservan como evidencia, mientras que la interpretación trabaja con una representación compacta. Con ello se equilibra la trazabilidad con los límites de contexto del modelo.

Por último, el sistema incorpora controles sobre el análisis generado y lo ejecuta en un proceso independiente, pero no constituye un entorno de aislamiento completo. El diseño reduce riesgos y limita operaciones, aunque no sustituye a una infraestructura de ejecución preparada para un servicio financiero en producción. Esta diferencia entre prototipo controlado y sistema productivo se mantiene explícita en el alcance del trabajo.

En conjunto, el diseño busca que la flexibilidad del LLM aparezca donde aporta valor: comprender peticiones diversas, adaptar el análisis y explicar resultados. Las operaciones críticas quedan rodeadas de información estructurada, validaciones y registros. El capítulo siguiente desarrolla cómo se materializan estas decisiones, mientras que el capítulo dedicado a la integración del modelo explica los contratos que guían a cada agente.

Capítulo 6

Implementación

El capítulo anterior presentó la arquitectura desde una perspectiva funcional. A continuación se explica cómo se han llevado esas decisiones a una aplicación capaz de recibir una consulta, mantener su estado, preparar los datos, realizar los cálculos y conservar los resultados. El propósito no es recorrer cada fichero del proyecto, sino describir los mecanismos técnicos que hacen posible revisar el flujo y conocer qué ha ocurrido en cada caso.

La aplicación se ha desarrollado en Python y separa sus responsabilidades principales. La orquestación coordina las etapas; los contratos definen qué información se intercambia; las utilidades de carga ofrecen una representación común de los CSV; y la capa de cálculo revisa el código generado antes de utilizar sus resultados. Así, cada componente puede examinarse de forma independiente y la lógica no queda concentrada en un único bloque difícil de mantener.

6.1. Organización técnica de la implementación

La aplicación se divide en varios bloques funcionales. El primero representa la entrada, el plan analítico, los resultados y el estado del caso. El segundo coordina el orden de las validaciones, las intervenciones del modelo y los cálculos. Otro bloque reúne la carga y normalización de los datos financieros. La configuración, los registros y las pruebas completan el entorno necesario para utilizar y revisar el prototipo.

Esta división responde a una necesidad práctica. La consulta del usuario cambia en cada caso, pero las reglas para validar fechas, localizar datos, registrar errores o limitar la ejecución permanecen estables. Mantener estas operaciones fuera de las intervenciones generativas reduce la variabilidad y permite comprobarlas mediante pruebas automatizadas.

Bloque funcional	Responsabilidad en la implementación
Estructuras y contratos	Representar la entrada, el plan, el estado, los resultados y los artefactos de ejecución.
Orquestación	Coordinar el recorrido, actualizar el estado y decidir cuándo continuar o finalizar.
Tratamiento de datos	Leer los CSV disponibles y transformarlos en una representación homogénea.
Validación y ejecución	Revisar el procedimiento generado, ejecutarlo con límites y recoger su salida.
Configuración y registros	Separar credenciales, parámetros, resultados y evidencias de cada caso.
Pruebas	Comprobar el comportamiento esperado sin depender de llamadas reales al modelo.

Tabla 6.1: Bloques funcionales de la implementación.

El repositorio refleja esta separación entre código, datos, documentación, pruebas y resultados. Su distribución física es secundaria: lo relevante es que cada bloque tenga una responsabilidad reconocible y que la información pase de una etapa a otra de forma explícita.

6.2. Orquestación y gestión del estado

El recorrido definido en el diseño se implementa como una secuencia de etapas conectadas. Cada una recibe el estado actual, realiza una operación concreta y lo devuelve actualizado. La información no circula mediante variables aisladas ni conversaciones acumuladas, sino a través de un registro común que acompaña al caso desde el inicio hasta su finalización.

El recorrido comienza con la validación de la entrada. Si la consulta y los datos son válidos, continúa hacia la planificación, la generación de código, su revisión, el cálculo y la interpretación. Después de cada paso se comprueba el estado alcanzado. Una entrada incorrecta conduce a una respuesta controlada; un problema durante la generación o validación impide ejecutar el código; y un fallo en los cálculos evita que sus resultados se presenten como correctos.

La orquestación principal utiliza un grafo de estados. Este enfoque permite expresar de forma explícita las transiciones normales y las rutas de error. Además, la implementación conserva una alternativa secuencial con el mismo orden de etapas. Esta segunda opción simplifica las pruebas y evita que el funcionamiento básico dependa por completo de una única biblioteca de orquestación.

Estado	Significado	Decisión asociada
Creado	La consulta ha sido recibida, pero todavía no se ha validado.	Iniciar las comprobaciones de entrada.
Validado	La petición contiene la información mínima y los datos existen.	Solicitar la planificación analítica.
Planificado	Se dispone de un plan estructurado.	Preparar el procedimiento de cálculo.
Preparado	El análisis ha sido generado y puede revisarse.	Aplicar los controles previos.
Ejecutado	El procedimiento terminó y produjo una salida.	Interpretar únicamente si la ejecución fue correcta.
Completado	Existe una respuesta final basada en resultados calculados.	Finalizar el caso.
Completado con error	Alguna etapa no pudo recuperarse.	Devolver el motivo conservado en el estado.

Tabla 6.2: Evolución simplificada del estado durante una ejecución.

La tabla presenta una lectura conceptual del recorrido y no reproduce literalmente todos los identificadores internos. En la aplicación se utilizan estados más específicos para distinguir, por ejemplo, una entrada inválida, un código rechazado o un cálculo fallido.

Este registro permite localizar el origen de un problema. Una consulta puede disponer de un plan correcto y fallar al generar el código, o completar los cálculos y encontrar el problema durante la redacción final. Esta diferencia resulta necesaria para evaluar el sistema por etapas y no únicamente por la apariencia de la respuesta.

6.3. Contratos y validación de entrada

La comunicación entre etapas utiliza contratos conocidos. La entrada reúne la consulta en lenguaje natural, los activos solicitados, el periodo o las fechas, el intervalo temporal, las rutas de los datos y los avisos disponibles. A medida que avanza el caso se añaden el plan analítico, el código generado, los resultados, los registros de error y la respuesta final.

El uso de contratos explícitos evita que una etapa tenga que interpretar libremente la salida de la anterior. También permite diferenciar los campos obligatorios de la

información opcional. Por ejemplo, una consulta puede utilizar un periodo relativo o un intervalo definido mediante fechas, pero siempre debe contener al menos un activo y un conjunto de datos disponible.

Antes de realizar una llamada al modelo se aplican varias comprobaciones deterministas:

- la consulta no puede estar vacía;
- debe haberse indicado al menos un activo;
- debe existir al menos un fichero de datos;
- las fechas, cuando se utilizan, deben respetar un formato común;
- las rutas recibidas deben corresponder a ficheros existentes.

Estas comprobaciones son sencillas, pero evitan que el modelo intente completar información que el sistema no posee. Cuando alguna condición no se cumple, el flujo termina antes de consumir recursos generativos y devuelve una explicación concreta sobre la información ausente o incorrecta.

El plan analítico y el resultado del cálculo también siguen formatos definidos. El primero identifica el objetivo, el nivel de profundidad, las métricas y los requisitos de salida. El segundo reúne las cifras, tablas, series y limitaciones obtenidas. El capítulo siguiente explica cómo interviene el modelo; en este punto interesa que la aplicación puede almacenar y trasladar estos contenidos entre etapas.

6.4. Carga y normalización de los datos

Los datos financieros utilizados por el prototipo proceden de CSV descargados previamente. Aunque todos contienen información histórica, su estructura no siempre es idéntica. Algunos ficheros presentan una tabla convencional con una fila de cabecera, mientras que otros conservan el formato ancho y las cabeceras multinivel generadas durante la descarga. Esta diferencia fue una fuente de errores en las primeras iteraciones.

Para no resolver de nuevo la lectura en cada consulta, se incorporó una capa común de carga. Esta identifica el formato disponible, extrae las columnas financieras y produce una tabla normalizada con fecha, activo y variables de mercado. Los campos principales son apertura, máximo, mínimo, cierre, cierre ajustado y volumen, siempre que estén disponibles.

Durante la normalización se convierten las fechas a un tipo temporal común y las columnas financieras a valores numéricos. Las filas que no permiten identificar una

fecha, un activo o un precio de cierre válido se descartan. Cuando la consulta solicita activos concretos, el resultado se filtra para conservar únicamente esos instrumentos. Si ninguno aparece en los datos, la carga termina con un error explícito.

La representación normalizada puede utilizarse de dos formas. Para análisis que requieren máximos, mínimos o volumen se conserva el formato largo, con una fila por fecha y activo. Para operaciones basadas únicamente en precios de cierre se genera una tabla en la que las fechas forman el índice y cada activo ocupa una columna. Esta segunda representación simplifica comparaciones, alineación temporal, rentabilidades y correlaciones.

La capa de datos también transforma tipos especiales en valores compatibles con JSON. Las fechas, los valores procedentes de librerías científicas y los resultados no finitos necesitan una conversión antes de incorporarse a la salida. Centralizar este tratamiento reduce fallos de serialización y evita repetir la misma lógica.

6.5. Ejecución controlada y tratamiento de resultados

Una vez revisado, el código se lanza en un proceso independiente de la aplicación principal. Recibe la consulta, los activos, las rutas de los CSV y el plan analítico. Esta entrada se conserva entre las evidencias del caso, de manera que pueda comprobarse qué información estaba disponible en el momento del cálculo.

El proceso se inicia con un tiempo máximo configurado, actualmente de 60 segundos. La biblioteca de ejecución interrumpe la espera cuando se supera ese límite. En la versión actual todavía conviene reforzar el tratamiento de esta excepción para convertirla siempre en un resultado interno uniforme, igual que ocurre con otros fallos. Cuando el proceso termina, la aplicación conserva por separado la salida producida, los mensajes de error y el código de finalización.

El resultado esperado es un objeto JSON. Cuando la salida respeta este formato, se transforma en una representación interna que puede utilizar la etapa siguiente. Si no puede analizarse, se conserva el texto original dentro de un campo de diagnóstico. Esta recuperación evita perder información, aunque no sustituye a una validación completa del esquema de salida, que sería una mejora conveniente.

Cada ejecución genera un conjunto mínimo de evidencias:

- el procedimiento que se intentó ejecutar;
- la entrada completa utilizada por ese procedimiento;
- la salida producida durante la ejecución;
- los mensajes de error;

- el código de finalización y el resultado interpretado.

Estas evidencias ayudan a reconstruir el caso y a comprobar si una cifra procede realmente de los cálculos. También facilitan distinguir un error del modelo de un problema en los datos o en las operaciones realizadas.

Cuando aparecen series largas, se conserva la salida completa y se prepara una versión reducida para la interpretación. Las listas extensas se sustituyen por una muestra del principio y del final, acompañada de su longitud original; los textos demasiado largos también se limitan. La reducción afecta únicamente al contexto enviado al modelo, no a la evidencia almacenada.

6.6. Validación, reparación y gestión de errores

El código generado no se utiliza de forma inmediata. Primero se comprueba que su sintaxis sea válida y que respete varias restricciones. Para ello se analiza su árbol sintáctico, con el que pueden identificarse importaciones y llamadas sin depender únicamente de búsquedas textuales.

La validación limita las bibliotecas que pueden utilizarse y bloquea operaciones ajenas al análisis previsto. También exige una estructura mínima y una salida serializada. Entre las operaciones no admitidas se encuentran la evaluación dinámica de expresiones, la creación de nuevos procesos, el acceso a red y la lectura libre de ficheros no incluidos en la entrada.

Comprobación	Finalidad
Sintaxis válida	Evitar la ejecución de un procedimiento que Python no puede interpretar.
Importaciones limitadas	Restringir el entorno a análisis de datos, operaciones matemáticas y serialización.
Operaciones bloqueadas	Impedir accesos o comportamientos que quedan fuera del alcance del prototipo.
Estructura mínima	Facilitar una ejecución uniforme y una revisión posterior.
Salida estructurada	Garantizar que los resultados puedan pasar a la siguiente etapa sin ambigüedad.

Tabla 6.3: Controles aplicados antes de la ejecución.

Si el código no supera la validación, el sistema puede solicitar una corrección. La nueva versión debe pasar exactamente por los mismos controles. Del mismo modo, si un código validado falla durante el cálculo, el error observado puede utilizarse para realizar un único intento de reparación y volver a probarlo.

La recuperación se mantiene acotada para evitar ciclos indefinidos y costes difíciles de comparar. Si la corrección tampoco cumple el contrato o vuelve a fallar, el caso termina con un error controlado. El estado conserva tanto el problema final como los avisos generados durante los intentos anteriores.

La validación reduce riesgos, pero no equivale a un aislamiento completo. El código se lanza en un proceso separado y con operaciones limitadas, aunque comparte el entorno de trabajo del prototipo. Una aplicación productiva requeriría medidas adicionales, como contenedores efímeros, permisos mínimos, límites de memoria y aislamiento de red.

6.7. Configuración, trazabilidad y reproducibilidad

Los parámetros del modelo y las credenciales se mantienen fuera del código mediante variables de entorno. Así puede documentarse la configuración necesaria sin almacenar las claves reales. Entre los valores configurables se encuentran el modelo, la dirección del servicio, el tiempo máximo de respuesta y algunos parámetros de generación.

La configuración local también determina el intérprete utilizado y el límite temporal de los cálculos. Mantener estos valores separados facilita adaptar el entorno sin alterar la lógica principal.

Los CSV empleados por la evaluación permanecen estables, mientras que el código y los registros se generan para cada caso. Los informes se construyen después a partir de esos resultados. Esta separación ayuda a reconstruir una prueba y evita confundir los datos de entrada con los productos obtenidos.

El control de versiones conserva la evolución del prototipo y de la memoria, mientras que los secretos, los entornos virtuales y los ficheros temporales quedan excluidos. El objetivo no es reproducir un despliegue productivo completo, sino dejar identificados los datos, la configuración y las evidencias necesarias para revisar las pruebas realizadas.

6.8. Pruebas automatizadas

Las pruebas automatizadas comprueban el recorrido principal sin depender de la disponibilidad del servicio LLM. Para ello se sustituyen temporalmente sus respuestas por planes, códigos e interpretaciones controladas. Así pueden verificarse la orquestación, la validación de entrada y los cálculos de manera rápida y repetible.

Los casos positivos cubren consultas de crecimiento, comparación, visión general, rentabilidad, riesgo y análisis técnico. En todos ellos se comprueba que el flujo alcanza el estado final esperado, que la ejecución termina correctamente y que la respuesta contiene información relacionada con los activos solicitados.

También se incluyen escenarios de error:

- ausencia de credenciales o configuración del modelo;
- fechas con un formato no admitido;
- ficheros de datos inexistentes;
- consultas vacías.

También existen comprobaciones unitarias sobre la lectura de la configuración y sobre algunos formatos de respuesta, como la detección de una interpretación devuelta únicamente en JSON. Sin embargo, la cobertura automatizada todavía no reproduce todas las respuestas mal formadas que puede generar el modelo ni todas las rutas de reparación. Estas situaciones se observan mediante la batería real y constituyen una línea clara para ampliar las pruebas.

Las pruebas automatizadas no sustituyen la evaluación con el modelo real. Su finalidad es comprobar que los componentes deterministas reaccionan de forma previsible y que los errores básicos conducen a una finalización controlada. La batería descrita en la metodología amplía esta verificación al comportamiento generativo.

En conjunto, la implementación transforma el diseño conceptual en un flujo operativo sin delegar todo el control en el modelo. Los contratos, la normalización de datos, las revisiones previas, el proceso separado y la conservación de evidencias forman la base técnica de los agentes LLM. El capítulo siguiente explica qué diferencia a cada uno y cómo se construyen sus instrucciones.

Capítulo 7

Integración del modelo de lenguaje

El modelo de lenguaje interviene en tres momentos del flujo: comprende la consulta, genera el código necesario y explica los resultados calculados. Aunque estas funciones se presentan como tres agentes, la implementación utiliza el mismo modelo con instrucciones, entradas y formatos de salida diferentes. Por tanto, no se trata de tres modelos entrenados por separado, sino de tres roles especializados dentro de una misma arquitectura.

Esta separación resulta útil porque las tres tareas persiguen objetivos distintos. Planificar exige entender la intención del usuario; generar código requiere precisión técnica; e interpretar resultados demanda claridad y prudencia. Reunirlas en una única llamada reduciría el número de interacciones, pero dificultaría revisar qué decisión se tomó en cada momento y dónde se originó un posible error.

7.1. Papel del modelo y división de responsabilidades

El modelo actúa como enlace entre el lenguaje natural y las partes deterministas del sistema. Se utiliza allí donde una regla fija resulta insuficiente: interpretar qué solicita el usuario, decidir qué métricas son pertinentes, adaptar el código al caso y redactar una explicación proporcionada.

En cambio, no funciona como fuente de precios ni como mecanismo de cálculo implícito. Tampoco puede descargar información, incorporar noticias, predecir movimientos futuros o formular recomendaciones de inversión. Las cifras que llegan al usuario deben proceder del código ejecutado sobre los CSV proporcionados.

Los agentes se diferencian por la transformación que realizan:

- el agente planificador convierte una petición abierta en un plan verificable;
- el agente generador convierte ese plan en código ejecutable;

- el agente interpretador convierte resultados estructurados en una explicación comprensible.

El primer agente concreta la intención; el segundo, el cálculo; y el tercero, la comunicación. Entre ellos se sitúan las validaciones y la ejecución descritas en el capítulo anterior.

Agente	Objetivo	Información recibida	Salida
Planificación	Concretar la intención del usuario.	Consulta, activos, periodo, rutas de datos y reglas de profundidad.	Plan analítico en JSON.
Generación de código	Traducir el plan a operaciones calculables.	Consulta original, entrada validada, rutas de datos, plan y contrato técnico.	JSON que contiene el código completo.
Interpretación	Comunicar los resultados obtenidos.	Plan y salida calculada, compactada cuando es necesario.	Texto en español, fiel y prudente.

Tabla 7.1: Diferencias principales entre los tres agentes.

La elección de estos tres roles responde a la decisión arquitectónica expuesta en el capítulo de diseño. No se busca simular un equipo de modelos diferentes, sino separar transformaciones que conviene poder revisar de forma independiente. El plan funciona como contrato entre la consulta y el código; los resultados ejecutados separan el cálculo de su explicación; y la respuesta final queda vinculada a unas evidencias concretas. Esta organización permite utilizar instrucciones más precisas para cada tarea y atribuir un posible fallo a la etapa en la que realmente se produjo.

7.2. Diseño común de los prompts

Los tres agentes comparten una instrucción de sistema que delimita el ámbito de actuación. El modelo debe comportarse como un componente de análisis financiero histórico y trabajar exclusivamente con la información proporcionada. Se prohíben los datos externos, las predicciones, el asesoramiento financiero y las recomendaciones de compra o venta.

El núcleo de esta instrucción puede resumirse en el siguiente fragmento:

Trabajas exclusivamente con datos históricos ya disponibles en CSV y con el contrato JSON indicado. No realizas predicciones, asesoramiento de inversión, re-

comendaciones de compra o venta ni incorporas información que no esté en la entrada.

La misma idea se adapta después a cada tarea. El planificador no debe calcular cifras; el generador no redacta la respuesta final; y el interpretador no puede recalcular ni completar valores ausentes. Las restricciones se repiten de forma deliberada para que cada llamada sea comprensible por sí misma y no dependa de que el modelo recuerde una interacción anterior.

Los prompts combinan cuatro elementos:

1. una definición clara del rol;
2. la información disponible para esa etapa;
3. el formato exacto que debe devolverse;
4. las operaciones permitidas y prohibidas.

En el cuerpo principal se recogen los fragmentos que mejor muestran estas diferencias. Las plantillas completas, sin los valores variables que se insertan en cada consulta, se incluyen en los anexos.

7.3. Prompts específicos de los agentes

7.3.1. Agente 1: planificación analítica

El primer agente recibe la consulta completa y la entrada estructurada. Su función consiste en decidir qué debe analizarse, no en realizar el cálculo. Debe identificar el objetivo financiero, seleccionar las métricas, indicar las columnas necesarias y anticipar cómo debería organizarse el resultado.

La instrucción principal establece:

Tu tarea es convertir la petición del usuario en un plan verificable para un código Python posterior. No calcules cifras y no redactes la respuesta final.

La salida es un objeto JSON con el nivel de análisis, el objetivo, las métricas, las columnas requeridas, las necesidades de datos, los requisitos de salida y las preferencias de presentación. También incluye una justificación breve. Esta última no pretende obtener una cadena de pensamiento extensa, sino dejar constancia de por qué se eligieron ese nivel y esas métricas.

El prompt evita objetivos vagos. En lugar de indicar simplemente que se realizará un análisis, el agente debe concretar si necesita rentabilidad, volatilidad, drawdown,

correlación, medias móviles, rankings temporales u otras medidas relacionadas con la consulta.

También diferencia las métricas de la forma de presentarlas. Una tabla comparativa o una serie normalizada no son cálculos por sí mismos, sino maneras de organizar los resultados. Esta separación ayuda a que el siguiente agente prepare una salida coherente.

7.3.2. Agente 2: generación de código

El segundo agente recibe la consulta original, la entrada validada, las rutas de los CSV y el plan analítico. No recibe los datos ya normalizados: la carga se realiza posteriormente, cuando se ejecuta el código mediante las utilidades comunes del proyecto.

Este agente está sometido al contrato más restrictivo. Su respuesta debe ser JSON y contener un único campo con el código completo. No se aceptan explicaciones, fragmentos parciales ni bloques Markdown. El fragmento central del prompt indica:

Devuelve exclusivamente un objeto JSON válido con un único campo `code`. El valor debe ser un código Python completo y ejecutable. Debe leer la entrada indicada, calcular solo con los CSV recibidos y escribir un único JSON como resultado.

Además, el código debe utilizar las funciones comunes de carga, limitarse a las bibliotecas autorizadas, evitar el acceso a red y controlar datos ausentes, divisiones por cero y problemas de serialización. Si una métrica no puede calcularse, debe devolver un valor nulo y explicar la limitación en lugar de inventar un resultado.

Las exigencias cambian según la profundidad del plan. Una consulta A suele necesitar las métricas esenciales y un resumen. Los niveles B y C pueden requerir tablas, comparaciones, rankings o datos preparados para una representación visual. Incluso en estos casos, la salida debe mantenerse procesable y evitar series innecesariamente extensas.

La diferencia respecto al agente interpretador es clara: el generador no escribe para el usuario. Su prioridad es producir un cálculo correcto, estructurado y revisable.

7.3.3. Reparación del código generado

Cuando el código no supera la validación o falla durante el cálculo, puede realizarse una llamada de reparación. Esta conserva el mismo rol, pero añade la versión anterior y el error observado.

El código anterior falló o fue rechazado. Devuelve una versión completa, no un parche parcial. Corrige la causa indicada y mantén el objetivo, el nivel y las salidas solicitadas por el plan.

La corrección no evita ningún control. La nueva versión vuelve a validarse y solo se ejecuta si cumple el mismo contrato que la primera. Como el modelo puede devolver otra respuesta incorrecta, el número de intentos se mantiene limitado y su resultado queda registrado.

7.3.4. Agente 3: interpretación de resultados

El tercer agente recibe el plan y la salida calculada. No vuelve a leer los CSV ni decide nuevas métricas. Su tarea consiste en seleccionar la información relevante, explicar su significado y adaptar la extensión a la petición.

Su instrucción principal cambia el formato exigido:

Redacta la respuesta final usando únicamente los resultados y el plan analítico. No recalculas, no completes cifras ausentes y no incorpores datos externos. Devuelve texto en español con frases completas, no JSON ni bloques de código.

La respuesta debe distinguir entre datos históricos observados, lectura interpretativa y limitaciones. Si existe una tabla o una serie preparada para una representación visual, puede explicar qué muestra, pero no debe afirmar que se ha generado una imagen. Tampoco puede ocultar valores nulos o datos insuficientes.

La extensión se adapta al nivel identificado durante la planificación. Una respuesta A debe ser breve y directa; una B puede organizar varias métricas y comparaciones; y una C debe comportarse como un informe compacto, con apartados claros cuando la consulta los requiera.

7.4. Niveles de profundidad A, B y C

Los niveles gradúan la complejidad sin cambiar de modelo ni utilizar tres flujos distintos. Orientan el plan, la salida del código y la redacción final.

Nivel	Tipo de petición	Resultado esperado	Respuesta final
A	Consulta directa o comparación sencilla.	Métricas esenciales y resumen del cálculo.	Breve y centrada en la pregunta.
B	Petición con varias métricas, tabla, serie o explicación ampliada.	Resultados estructurados y comparación clara.	Organizada y con contexto suficiente.
C	Informe profesional, multicriterio o con formato impuesto.	Varias familias de métricas, tablas, datos visuales y limitaciones.	Informe compacto con apartados diferenciados.

Tabla 7.2: Adaptación del comportamiento según el nivel de profundidad.

La clasificación no depende únicamente del número de activos. Una consulta sobre un solo instrumento puede pertenecer al nivel C si exige varias familias de riesgo, datos de evolución y una organización concreta. Del mismo modo, una comparación entre dos activos puede mantenerse en el nivel A si solo pregunta cuál obtuvo mayor rentabilidad.

El prompt utiliza señales como la solicitud de tablas, series normalizadas, draw-down, medias móviles, rankings, periodos extremos o bloques obligatorios. Estas referencias ayudan a mantener un criterio estable, aunque la decisión final debe atender al conjunto de la petición.

7.5. Procesamiento de respuestas y gestión de errores

Las respuestas del planificador y del generador deben poder procesarse automáticamente. El sistema elimina envoltorios de formato cuando aparecen y busca un objeto JSON válido. Si encuentra texto adicional alrededor, intenta recuperar únicamente el fragmento estructurado.

Cuando la respuesta no puede analizarse, se realiza un nuevo intento con una indicación más directa: devolver exclusivamente JSON válido, sin explicaciones ni Markdown. El número de intentos está limitado. En la generación de código también se admite una recuperación acotada cuando el modelo devuelve directamente un programa reconocible, aunque no haya respetado por completo el envoltorio solicitado.

La interpretación sigue la lógica contraria. Su resultado debe ser texto natural. Si el modelo devuelve únicamente JSON, se solicita una nueva respuesta que explique los valores mediante frases completas. Esto refleja una decisión central: la estructura se reserva para coordinar las etapas internas y el texto libre para la comunicación final.

Los errores pueden aparecer en cualquiera de los roles. El planificador puede devolver un formato incompleto o seleccionar métricas poco adecuadas; el generador puede

producir JSON inválido, código con errores sintácticos u operaciones rechazadas; y el interpretador puede devolver texto vacío, repetir la estructura recibida o redactar una explicación insuficiente.

La aplicación conserva el punto en el que apareció el problema. Así puede distinguirse un fallo de comunicación con el servicio, una respuesta mal formada, un rechazo técnico, un error durante los cálculos o una interpretación que no llegó a completarse. Los resultados concretos de estos comportamientos se presentan en el capítulo de evaluación.

7.6. Cliente, modelo y configuración

La integración utiliza un cliente compatible con la interfaz de mensajes de Chat Completions. Cada petición se representa mediante una secuencia de mensajes con distintos roles y el modelo devuelve una respuesta asociada a esa conversación [25]. El sistema construye una conversación independiente para cada agente, formada por la instrucción común y el mensaje específico de la tarea.

El modelo evaluado es `openai/gpt-oss-20b`, un modelo de pesos abiertos documentado por OpenAI [6]. Durante las pruebas se accede a él mediante un servidor vLLM compatible con la interfaz del cliente. Conviene separar ambos conceptos: el modelo aporta la capacidad generativa y vLLM proporciona la infraestructura de inferencia.

La configuración permite indicar la dirección del servicio, el nombre del modelo, el tiempo máximo de respuesta, la temperatura y el límite de tokens. Se utiliza una temperatura baja para reducir variaciones innecesarias, aunque el comportamiento continúa siendo generativo y una única ejecución no elimina la posibilidad de obtener respuestas distintas.

7.7. Alcance de la estrategia de prompts

Los prompts se han ajustado de forma incremental a partir de los problemas encontrados durante el desarrollo. Los formatos estructurados, las utilidades comunes de carga, las reglas de serialización y los recordatorios sobre datos insuficientes responden a dificultades observadas en las primeras pruebas.

Estas instrucciones mejoran la estabilidad, pero no garantizan que todas las respuestas sean correctas. Un prompt orienta al modelo; no constituye una verificación formal. Por esa razón, las salidas se combinan con controles deterministas y se conservan los productos intermedios.

La división en tres agentes tampoco es la única arquitectura posible. Una sola llamada reduciría coste y latencia, pero ofrecería menos puntos de control y ocultaría

parte del recorrido seguido por el modelo. Para el propósito de este trabajo, la separación resulta adecuada porque permite evaluar de forma diferenciada la comprensión, la generación de código y la redacción, además de localizar con mayor precisión el origen de los fallos.

En conjunto, la integración busca aprovechar la flexibilidad del LLM sin convertirlo en la única fuente de control. Los prompts delimitan cada tarea, los contratos conectan las etapas y la validación externa decide si sus productos pueden utilizarse.

Capítulo 8

Resultados y evaluación

8.1. Objetivo

La evaluación comprueba si el prototipo mantiene el flujo completo al aumentar la dificultad de las consultas. No busca demostrar capacidad predictiva ni establecer un ranking de servicios LLM. El foco está en verificar planificación, generación de código, validación, ejecución e interpretación final sobre datos históricos locales.

8.2. Batería A/B/C

La batería principal contiene 15 consultas equilibradas: cinco de Nivel A, cinco de Nivel B y cinco de Nivel C. La dificultad asciende dentro de cada nivel.

Nivel	Progresión de dificultad	Comprobación principal
A	Crecimiento simple, comparación básica, visión descriptiva, retornos y riesgo básico.	Que el sistema responda consultas directas sin sobrecargar la salida.
B	Análisis ampliado, comparación normalizada, retorno-riesgo, activo volátil e intradía.	Que el sistema produzca métricas y estructura tabular o visual cuando se pide más detalle.
C	Informe comparativo, ranking multicriterio, índice amplio, materia prima intradía y formato de estrés.	Que el sistema coordine varias métricas y restricciones simultáneas.

Tabla 8.1: Estructura de la batería A/B/C de 15 consultas.

La quinta consulta de Nivel C fuerza cuatro bloques de salida: resumen de métricas, clasificación mensual, datos normalizados y datos de drawdown. Su objetivo es presionar la planificación y el código generado, no producir una figura en la memoria.

8.3. Métricas técnicas

La evaluación se apoya en métricas técnicas registrables:

- **Compleitud:** estado final `completed`.
- **Plan generado:** existencia de un plan A/B/C parseable.
- **Código generado:** existencia de script que cumple el contrato mínimo.
- **Código validado:** superación de la revisión de seguridad.
- **Ejecución correcta:** retorno cero y salida JSON parseable.
- **Interpretación final:** respuesta en español basada en resultados calculados.
- **Prudencia financiera:** ausencia de recomendaciones de compra o venta.
- **Tiempo de ejecución:** duración observada por caso.

Las métricas subjetivas, como claridad percibida, utilidad o calidad comunicativa, quedan previstas para una fase posterior. Se consideran relevantes, pero deben reforzarse con una rúbrica más estable y revisores consistentes antes de tratarlas como evidencia principal.

8.4. Resultados observados

La versión actual del prototipo se orienta a `openai/gpt-oss-20b` servido sobre vLLM. vLLM se interpreta como el servidor de despliegue compatible, no como un proveedor alternativo del estudio.

La evaluación registra para cada caso si el flujo llega a planificar, generar código, validar, ejecutar e interpretar. Los casos parciales son informativos porque permiten localizar el punto de fallo. Por ejemplo, un caso con plan correcto pero ejecución fallida señala un problema distinto a un caso que no llega a producir JSON válido en la primera llamada LLM.

Resultado registrado	Interpretación
<code>completed</code>	El flujo alcanza la respuesta final con salida ejecutada.
<code>completed_with_error</code>	El sistema devuelve un error controlado y no inventa una respuesta alternativa.
<code>code_rejected</code>	El script no supera el contrato o la validación de seguridad.
<code>execution_failed</code>	El script era ejecutable pero falló al calcular o serializar la salida.
Avisos de reparación	El sistema necesitó una segunda llamada para corregir código o formato.

Tabla 8.2: Lectura de los estados principales de evaluación.

8.5. Limitaciones

Una única pasada de evaluación no permite afirmar robustez estadística. La disponibilidad del modelo, la cuota, la latencia y la estabilidad del endpoint pueden afectar a los resultados. Además, la generación de código mediante LLM requiere validación estricta porque una solución plausible puede fallar por detalles de formato, columnas, datos insuficientes o serialización.

El sistema trabaja con datos históricos locales. No incorpora noticias, datos fundamentales ni precios actuales externos. Por tanto, las respuestas deben leerse como análisis descriptivos del periodo disponible, no como predicciones ni recomendaciones de inversión.

8.6. Síntesis

La evaluación muestra un prototipo funcional y trazable. La contribución principal es que el sistema no mide sólo si el texto final parece útil, sino si cada fase produce artefactos verificables: plan, código, validación, ejecución y explicación. La batería A/B/C permite aumentar la exigencia de forma ordenada y deja preparadas métricas subjetivas que podrán reforzarse en trabajo futuro.

Capítulo 9

Aspectos éticos, seguridad y uso responsable

9.1. Enfoque general

El uso responsable no se entiende en este trabajo como una advertencia añadida al final del desarrollo, sino como una condición que afecta a todo el recorrido del sistema. La consulta se acota antes de procesarse, el modelo trabaja mediante contratos definidos, el código se revisa antes de ejecutarse y la respuesta final conserva las limitaciones del análisis. Aun así, ninguna de estas medidas elimina por completo el riesgo de producir un cálculo incorrecto o una explicación inadecuada.

Este planteamiento coincide con los marcos actuales de gestión del riesgo en inteligencia artificial, que recomiendan considerar la fiabilidad, la transparencia y la supervisión durante todo el ciclo de vida, y no únicamente al evaluar la salida final [26]. El Reglamento europeo de inteligencia artificial adopta también un enfoque basado en el riesgo y concede especial importancia a la supervisión humana, la robustez técnica y la información clara sobre las capacidades y limitaciones del sistema [27]. Estas referencias se utilizan como criterios de diseño; no se pretende establecer en este capítulo una clasificación jurídica del prototipo, que dependería de su finalidad y de las condiciones concretas de un posible despliegue.

9.2. Finalidad prevista y usos excluidos

El sistema está diseñado para responder preguntas sobre series financieras históricas ya disponibles. Puede calcular rentabilidad, volatilidad, drawdown, correlaciones, medias móviles, extremos u otras medidas descriptivas, y organizar los resultados en tablas o series estructuradas. Su finalidad es facilitar el análisis y mostrar de forma trazable cómo se ha llegado a una respuesta.

Quedan fuera de su alcance la predicción de precios, la recomendación de compra o venta, la gestión de carteras, la adaptación a la situación personal de un inversor y la ejecución de operaciones en el mercado. El sistema tampoco decide qué activo debe descargarse ni busca información por su cuenta. Trabaja con instrumentos identificados previamente y con rutas de datos proporcionadas en la entrada.

Esta delimitación es especialmente importante porque una respuesta bien redactada puede transmitir más seguridad de la que justifican los datos. Las métricas históricas describen lo ocurrido durante un periodo concreto, pero no garantizan que el comportamiento vaya a repetirse. Las advertencias dirigidas a inversores insisten precisamente en que las afirmaciones sobre resultados pasados deben presentarse con contexto y sin inducir expectativas injustificadas [28].

Uso previsto	Uso no previsto
Análisis descriptivo de datos históricos seleccionados.	Predicción de precios o rentabilidades futuras.
Cálculo y explicación de métricas financieras.	Recomendaciones de inversión o asesoramiento personalizado.
Comparación trazable de activos y periodos.	Ejecución automática de órdenes o gestión de una cartera.
Apoyo a la revisión académica del flujo.	Sustitución de una validación financiera profesional.

Tabla 9.1: Delimitación del uso responsable del sistema.

9.3. Interpretación financiera y supervisión humana

El principal riesgo para el usuario no es únicamente que una cifra sea incorrecta. También puede ocurrir que un dato correcto se presente sin el contexto necesario, que se confunda una relación histórica con una explicación causal o que una respuesta fluida se acepte sin revisión. Esta tendencia a confiar en exceso en una salida automatizada resulta especialmente delicada cuando el contenido se refiere a decisiones económicas.

El diseño intenta reducir este riesgo separando planificación, cálculo e interpretación. El agente final no recibe libertad para reconstruir el análisis: trabaja con el plan y con la salida obtenida al ejecutar el código. Además, se le pide distinguir entre datos observados, lectura interpretativa y limitaciones, y se prohíbe incorporar información externa o completar cifras ausentes. Esta separación facilita comprobar si la redacción se corresponde con los resultados.

No obstante, los tres agentes utilizan el mismo modelo. Por ello, la división funcional

mejora la trazabilidad, pero no equivale a contar con tres revisores independientes. Un supuesto equivocado puede propagarse desde el plan hasta el código y conservar una apariencia coherente en la respuesta final. La revisión humana sigue siendo necesaria para confirmar la identidad del instrumento, la adecuación de las métricas, las unidades, el periodo analizado y la interpretación de los resultados.

La supervisión debe ser proporcional al uso. En el contexto académico resulta suficiente conservar los artefactos y revisar los casos relevantes. En una aplicación dirigida a usuarios externos sería necesario mostrar con claridad que la respuesta ha sido generada con ayuda de inteligencia artificial, facilitar el acceso a sus fuentes y limitaciones, y permitir que una persona descarte el resultado cuando detecte una anomalía.

9.4. Calidad, procedencia y alcance de los datos

La calidad de la respuesta está condicionada por la calidad de los datos de entrada. La batería utiliza series descargadas previamente y almacenadas de forma local. Esta decisión favorece la reproducibilidad y evita que el sistema elija automáticamente un instrumento distinto del solicitado, pero también limita su autonomía y hace necesario preparar las fuentes antes de formular la consulta.

La identificación del activo no siempre es trivial. Una expresión como “oro” puede aludir a un precio de referencia, a un contrato de futuros o a un fondo cotizado. Del mismo modo, dos series sobre un mismo mercado pueden diferir por moneda, horario, ajustes o calendario. El sistema no resuelve por sí solo estas ambigüedades; presupone que el conjunto de datos asociado a la consulta ha sido seleccionado correctamente.

Los datos proceden de descargas realizadas mediante `yfinance`, una herramienta abierta que no está afiliada ni respaldada por Yahoo [20], [21]. El prototipo tampoco incorpora noticias, información fundamental, variables macroeconómicas, costes de transacción ni datos en tiempo real. En comparaciones entre mercados diferentes no siempre se homogeneizan monedas y calendarios. Estas ausencias no invalidan un análisis descriptivo, pero sí restringen las conclusiones que pueden extraerse.

La batería define los componentes esperados de cada respuesta, aunque no contiene una referencia numérica validada manualmente para todas las métricas. En consecuencia, completar el flujo demuestra que el sistema ha podido planificar, generar, ejecutar e interpretar una solución, pero no certifica por sí solo la corrección financiera de cada cifra. Para un uso de mayor impacto sería necesario contrastar los cálculos con implementaciones de referencia y añadir controles específicos para cada familia de métricas.

9.5. Seguridad del código generado

El código producido por un LLM debe considerarse una entrada no fiable. OWASP identifica como riesgos relevantes tanto el tratamiento insuficiente de las salidas del modelo como la concesión de más funciones, permisos o autonomía de los necesarios [29], [30]. En este sistema, el modelo puede proponer un programa, pero no decide directamente si debe ejecutarse.

La validación analiza el árbol sintáctico del código y exige una función principal y una salida JSON. Se limita el conjunto de importaciones y se rechazan llamadas como `eval`, `exec`, `compile`, `open` o `__import__`. También se bloquean módulos asociados a procesos, red y manipulación libre del sistema de archivos. Los prompts refuerzan estas condiciones al exigir las funciones de carga previstas y las rutas recibidas en la entrada.

Si el código supera esta revisión, se ejecuta en un proceso independiente con un límite de 60 segundos. Se conservan el programa, el payload, la salida estándar, la salida de error y el código de retorno. Cuando aparece un rechazo o un fallo de ejecución, puede realizarse un único intento de reparación; la nueva versión vuelve a pasar por la misma validación antes de ejecutarse. Si no supera los controles, el flujo termina con un error explícito y no fabrica una respuesta financiera alternativa.

Estas medidas reducen la superficie de riesgo, pero no constituyen una sandbox completa. El proceso comparte el entorno de trabajo, hereda variables del proceso principal y no dispone de límites específicos de memoria o CPU. La revisión estática tampoco demuestra que el cálculo sea correcto ni garantiza que todas las formas posibles de acceso a recursos queden cubiertas. Por este motivo, el prototipo no debe ejecutar código generado a partir de entradas no confiables en un entorno expuesto.

Riesgo	Control actual	Riesgo residual
Código sintácticamente inválido	Análisis previo y reparación limitada.	La reparación también puede fallar.
Uso de operaciones peligrosas	Lista de importaciones y llamadas bloqueadas.	La revisión estática no equivale a aislamiento.
Ejecución indefinida	Proceso separado y límite temporal.	No hay cuotas específicas de memoria o CPU.
Salida no interpretable	Contrato JSON y registro de <code>stdout</code> y <code>stderr</code> .	Un JSON válido puede contener cálculos equivocados.
Respuesta no respaldada	Interpretación basada en el plan y la salida ejecutada.	El modelo puede omitir contexto o expresar mal una cifra.

Tabla 9.2: Controles de seguridad y riesgos que permanecen abiertos.

9.6. Trazabilidad y rendición de cuentas

Cada ejecución conserva la entrada, el plan, el código, el resultado, los avisos, los errores y la respuesta final. Esta información permite reconstruir el recorrido y atribuir un fallo a la etapa en la que se produjo. La trazabilidad no convierte automáticamente el sistema en fiable, pero evita que la única evidencia sea un texto final convincente.

La batería añade una preevaluación heurística de cinco dimensiones para facilitar la revisión. Estas puntuaciones ayudan a localizar casos problemáticos, aunque no sustituyen el criterio humano. Algunas dimensiones se aproximan mediante la presencia de términos, la longitud de la respuesta o el estado del flujo, de modo que una puntuación alta no demuestra que cada cálculo sea correcto.

En un despliegue real sería necesario definir quién responde de la selección de datos, quién aprueba las métricas, quién revisa las incidencias y durante cuánto tiempo se conservan los registros. También deberían versionarse el modelo, los prompts, las dependencias y el catálogo de datos para poder reproducir una respuesta concreta.

9.7. Privacidad, confidencialidad y credenciales

Los conjuntos utilizados contienen precios y volúmenes históricos, no información personal. Las claves de acceso al servicio LLM se almacenan en el fichero local `.env`, excluido del control de versiones. Los directorios que contienen código generado, payloads, registros e informes de evaluación también se excluyen del repositorio.

Sin embargo, las llamadas al modelo transmiten la consulta, información estructurada sobre los activos, las rutas de los datos, el plan y, en la fase final, los resultados calculados. Además, los registros locales conservan una parte importante de ese contenido. En la batería actual esta información no es sensible, pero podría serlo si una versión futura aceptara carteras, estrategias internas o datos identificables de usuarios.

En ese escenario sería necesario aplicar minimización de datos, sustituir rutas locales por identificadores neutros, establecer plazos de conservación, controlar el acceso a los registros y documentar las condiciones de tratamiento del proveedor. Si se incorporasen datos personales, estos requisitos deberían alinearse con los principios de limitación de finalidad, minimización, exactitud e integridad establecidos por el Reglamento General de Protección de Datos [31].

9.8. Coste computacional y sostenibilidad

El trabajo no entrena ni ajusta un modelo propio. Reutiliza un modelo ya desplegado y reserva las llamadas generativas para tres funciones concretas: planificación,

generación de código e interpretación. Los cálculos financieros se realizan localmente sobre datos almacenados.

Esto no significa que el coste ambiental sea despreciable. Cada caso requiere normalmente varias inferencias y puede añadir una llamada de reparación. La infraestructura que sirve el modelo consume recursos que no se han medido en este estudio, por lo que no sería riguroso afirmar una reducción cuantificada de energía o emisiones.

Las medidas adoptadas son principalmente operativas: temperatura baja, límites de salida y tiempo, compactación del contexto enviado al agente final, reparación acotada y posibilidad de continuar una batería sin repetir casos ya completados. Estas decisiones contienen el consumo innecesario y el coste económico, aunque una evaluación ambiental sólida exigiría medir energía, hardware, duración y carga del servicio.

9.9. Condiciones para un uso futuro

El prototipo es adecuado como demostrador académico y como entorno de evaluación de un flujo LLM trazable. No está preparado para actuar como herramienta profesional autónoma. Antes de ofrecerlo a usuarios externos sería necesario, al menos, incorporar aislamiento real para el código, permisos mínimos, límites de recursos, gestión formal de registros y credenciales, validaciones financieras de referencia y revisión humana en los casos de mayor impacto.

También debería mantenerse una separación clara entre análisis y acción. Conectar el sistema a una cuenta de inversión, permitirle enviar órdenes o personalizar decisiones a partir de datos económicos de una persona cambiaría sustancialmente el nivel de riesgo y exigiría una evaluación jurídica, técnica y organizativa específica.

La conclusión práctica es deliberadamente prudente: el sistema puede ayudar a organizar y explicar un análisis histórico, pero la responsabilidad de comprobar los datos, interpretar el alcance y decidir si el resultado es utilizable continúa siendo humana.

Capítulo 10

Conclusiones y trabajo futuro

Referencias

- [1] A. Vaswani et al., «Attention Is All You Need,» en *Advances in Neural Information Processing Systems*, 2017. dirección: <https://arxiv.org/abs/1706.03762>
- [2] Analytics Vidhya, *60+ Generative AI Terms You Must Know*, 2024. visitado 25 de mayo de 2026. dirección: <https://www.analyticsvidhya.com/blog/2024/01/generative-ai-terms/>
- [3] D. Araci, «FinBERT: Financial Sentiment Analysis with Pre-trained Language Models,» *arXiv preprint arXiv:1908.10063*, 2019. DOI: 10.48550/arXiv.1908.10063 arXiv: 1908.10063 [cs.CL]. dirección: <https://arxiv.org/abs/1908.10063>
- [4] S. Wu et al., «BloombergGPT: A Large Language Model for Finance,» *arXiv preprint arXiv:2303.17564*, 2023. DOI: 10.48550/arXiv.2303.17564 arXiv: 2303.17564 [cs.LG]. dirección: <https://arxiv.org/abs/2303.17564>
- [5] H. Yang, X.-Y. Liu y C. D. Wang, «FinGPT: Open-Source Financial Large Language Models,» *arXiv preprint arXiv:2306.06031*, 2023. DOI: 10.48550/arXiv.2306.06031 arXiv: 2306.06031 [q-fin.CP]. dirección: <https://arxiv.org/abs/2306.06031>
- [6] OpenAI, *gpt-oss-120b & gpt-oss-20b Model Card*, 2025. DOI: 10.48550/arXiv.2508.10925 arXiv: 2508.10925 [cs.CL]. dirección: <https://arxiv.org/abs/2508.10925>
- [7] T. Zhou, P. Wang, Y. Wu y H. Yang, «FinRobot: AI Agent for Equity Research and Valuation with Large Language Models,» *arXiv preprint arXiv:2411.08804*, 2024. DOI: 10.48550/arXiv.2411.08804 arXiv: 2411.08804 [q-fin.CP]. dirección: <https://arxiv.org/abs/2411.08804>
- [8] Y. Xiao, E. Sun, D. Luo y W. Wang, «TradingAgents: Multi-Agents LLM Financial Trading Framework,» *arXiv preprint arXiv:2412.20138*, 2024. DOI: 10.48550/arXiv.2412.20138 arXiv: 2412.20138 [q-fin.TR]. dirección: <https://arxiv.org/abs/2412.20138>

- [9] S. Yao et al., «ReAct: Synergizing Reasoning and Acting in Language Models,» en *International Conference on Learning Representations*, 2023. arXiv: 2210.03629 [cs.CL]. dirección: <https://arxiv.org/abs/2210.03629>
- [10] T. Schick et al., «Toolformer: Language Models Can Teach Themselves to Use Tools,» *arXiv preprint arXiv:2302.04761*, 2023. DOI: 10.48550/arXiv.2302.04761 arXiv: 2302.04761 [cs.CL]. dirección: <https://arxiv.org/abs/2302.04761>
- [11] L. Wang et al., «A Survey on Large Language Model based Autonomous Agents,» *arXiv preprint arXiv:2308.11432*, 2023. DOI: 10.48550/arXiv.2308.11432 arXiv: 2308.11432 [cs.AI]. dirección: <https://arxiv.org/abs/2308.11432>
- [12] LangChain, «LangGraph: Building Stateful, Multi-Actor Applications with LLMs,» 2024. visitado 6 de jun. de 2026. dirección: <https://langchain-ai.github.io/langgraph/>
- [13] M. Chen et al., «Evaluating Large Language Models Trained on Code,» *arXiv preprint arXiv:2107.03374*, 2021. DOI: 10.48550/arXiv.2107.03374 arXiv: 2107.03374 [cs.LG]. dirección: <https://arxiv.org/abs/2107.03374>
- [14] Y. Lai et al., «DS-1000: A Natural and Reliable Benchmark for Data Science Code Generation,» *arXiv preprint arXiv:2211.11501*, 2022. DOI: 10.48550/arXiv.2211.11501 arXiv: 2211.11501 [cs.SE]. dirección: <https://arxiv.org/abs/2211.11501>
- [15] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt y R. Karri, «Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions,» en *2022 IEEE Symposium on Security and Privacy*, IEEE, 2022, págs. 754-768. DOI: 10.1109/SP46214.2022.9833571 dirección: <https://arxiv.org/abs/2108.09293>
- [16] Z. Ji et al., «Survey of Hallucination in Natural Language Generation,» *ACM Computing Surveys*, 2022. DOI: 10.1145/3571730 arXiv: 2202.03629 [cs.CL]. dirección: <https://arxiv.org/abs/2202.03629>
- [17] P. Lewis et al., «Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,» en *Advances in Neural Information Processing Systems*, 2020. arXiv: 2005.11401 [cs.CL]. dirección: <https://arxiv.org/abs/2005.11401>
- [18] P. Liang et al., «Holistic Evaluation of Language Models,» *arXiv preprint arXiv:2211.09110*, 2022. DOI: 10.48550/arXiv.2211.09110 arXiv: 2211.09110 [cs.CL]. dirección: <https://arxiv.org/abs/2211.09110>

- [19] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu y C. Zhu, «G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment,» en *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2023, págs. 2511-2522. DOI: 10.18653/v1/2023.emnlp-main.153 dirección: <https://aclanthology.org/2023.emnlp-main.153/>
- [20] R. Aroussi, *yfinance Python Library*, 2026. visitado 25 de mayo de 2026. dirección: <https://github.com/ranaroussi/yfinance>
- [21] Yahoo Finance Help, *Download historical data in Yahoo Finance*, 2026. visitado 25 de mayo de 2026. dirección: <https://help.yahoo.com/kb/finance/historical-prices-adjusted-close-sln2311.html>
- [22] London Bullion Market Association, *LBMA Gold Price*, 2026. visitado 7 de jun. de 2026. dirección: <https://www.lbma.org.uk/prices-and-data/lbma-gold-price>
- [23] CME Group, *Gold Product Overview*, 2026. visitado 7 de jun. de 2026. dirección: <https://www.cmegroup.com/education/courses/event-contracts-underlying-markets/product-gold.html>
- [24] State Street Global Advisors, *SPDR Gold Shares*, 2026. visitado 7 de jun. de 2026. dirección: <https://www.ssga.com/us/en/individual/etfs/spdr-gold-shares-gld>
- [25] OpenAI, *Chat Completions API Reference*, 2026. visitado 4 de jun. de 2026. dirección: <https://platform.openai.com/docs/api-reference/chat/create-chat-completion>
- [26] C. Autio et al., «Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile,» National Institute of Standards y Technology, inf. téc. NIST AI 600-1, 2024. DOI: 10.6028/NIST.AI.600-1 visitado 7 de jun. de 2026. dirección: <https://doi.org/10.6028/NIST.AI.600-1>
- [27] European Parliament and Council of the European Union, *Regulation (EU) 2024/1689 Laying Down Harmonised Rules on Artificial Intelligence*, 2024. visitado 7 de jun. de 2026. dirección: <https://eur-lex.europa.eu/eli/reg/2024/1689/oj>
- [28] U.S. Securities and Exchange Commission, *Investor Bulletin: Performance Claims*, 2022. visitado 27 de mayo de 2026. dirección: <https://www.investor.gov/introduction-investing/general-resources/news-alerts/alerts-bulletins/investor-bulletins-47>
- [29] OWASP Gen AI Security Project, *LLM05:2025 Improper Output Handling*, 2025. visitado 7 de jun. de 2026. dirección: <https://genai.owasp.org/llmrisk/llm05-supply-chain-vulnerabilities/>

- [30] OWASP Gen AI Security Project, *LLM06:2025 Excessive Agency*, 2025. visitado 7 de jun. de 2026. dirección: <https://genai.owasp.org/llmrisk/llm062025-excessive-agency/>
- [31] European Parliament and Council of the European Union, *Regulation (EU) 2016/679 on the Protection of Natural Persons with Regard to the Processing of Personal Data*, 2016. visitado 7 de jun. de 2026. dirección: <https://eur-lex.europa.eu/eli/reg/2016/679/oj>
- [32] J.-B. Alayrac et al., «Flamingo: A Visual Language Model for Few-Shot Learning,» en *Advances in Neural Information Processing Systems*, 2022. dirección: <https://arxiv.org/abs/2204.14198>
- [33] OpenAI, *GPT-4V(ision) System Card*, 2023. visitado 3 de jun. de 2026. dirección: <https://openai.com/index/gpt-4v-system-card/>
- [34] AI@Meta, «The Llama 3 Herd of Models,» *arXiv preprint arXiv:2407.21783*, 2024. DOI: 10.48550/arXiv.2407.21783 arXiv: 2407.21783 [cs.AI]. dirección: <https://arxiv.org/abs/2407.21783>
- [35] OpenAI, *OpenAI o1 System Card*, 2024. DOI: 10.48550/arXiv.2412.16720 arXiv: 2412.16720 [cs.AI]. dirección: <https://arxiv.org/abs/2412.16720>
- [36] DeepSeek-AI, «DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning,» *arXiv preprint arXiv:2501.12948*, 2025. DOI: 10.48550/arXiv.2501.12948 arXiv: 2501.12948 [cs.CL]. dirección: <https://arxiv.org/abs/2501.12948>
- [37] Investor.gov, *What is Risk?* 2026. visitado 27 de mayo de 2026. dirección: <https://www.investor.gov/introduction-investing/investing-basics/what-risk>
- [38] CFA Institute, *Portfolio Risk and Return: Part I*, 2026. visitado 27 de mayo de 2026. dirección: <https://www.cfainstitute.org/insights/professional-learning/refresher-readings/2026/portfolio-risk-return-part-1>
- [39] CFA Institute, *Performance Drawdowns in Asset Management: Extending Draw-down Analysis to Active Returns*, 2018. visitado 27 de mayo de 2026. dirección: <https://rpc.cfainstitute.org/research/cfa-digest/2018/01/dig-v48-n1-3>
- [40] Atlassian, *A Complete Guide to Line Charts*, 2026. visitado 27 de mayo de 2026. dirección: <https://www.atlassian.com/data/charts/line-chart-complete-guide>

Apéndice A

Anexos

A.1. Ejecución de la batería

La evaluación actual se realiza mediante `scripts/evaluate_bateria_50.py`, que centraliza la selección de casos, la ejecución del flujo y el almacenamiento de los resultados.

Antes de iniciar las llamadas al modelo puede comprobarse la selección sin ejecutar ningún caso:

```
python scripts/evaluate_bateria_50.py --list-only
python scripts/evaluate_bateria_50.py --levels C --list-only
```

La primera orden muestra los 50 identificadores y su distribución; la segunda permite revisar únicamente un nivel. Esta comprobación resulta útil para detectar filtros o nombres incorrectos antes de consumir tiempo de inferencia.

A.1.1. Ejecución completa

La batería entera puede lanzarse con una sola orden:

```
python scripts/evaluate_bateria_50.py
```

Esta opción es válida cuando el servicio permanece estable durante toda la ejecución. Sin embargo, cada caso puede requerir varias llamadas al modelo y, en algunos supuestos, un intento adicional de reparación. Por ello, una ejecución única prolongada queda más expuesta a cortes de conexión, límites temporales o interrupciones del proceso.

A.1.2. Ejecución por niveles

Para la evaluación presentada en este trabajo se utilizó una ejecución progresiva. Primero se procesaron los diez casos A; después se cargó ese resultado y se añadieron los veinte B; por último, se incorporaron los veinte C. Los comandos empleados fueron los siguientes:

```
python scripts/evaluate_bateria_50.py --levels A

$A = "results/evaluations/bateria_50_20260607_135100.json"
python scripts/evaluate_bateria_50.py --levels B '
--resume-json $A --skip-existing

$B = "results/evaluations/bateria_50_20260607_140922.json"
python scripts/evaluate_bateria_50.py --levels C '
--resume-json $B --skip-existing
```

La opción `-resume-json` recupera los resultados anteriores y los incorpora al nuevo fichero. A su vez, `-skip-existing` evita volver a ejecutar un caso que ya esté presente. Esta combinación permite continuar el proceso sin perder los niveles completados ni duplicar llamadas.

Después de terminar el nivel C se generó una versión consolidada cuya selección declarada abarca los 50 casos:

```
$C = "results/evaluations/bateria_50_20260607_145322.json"
python scripts/evaluate_bateria_50.py '
--resume-json $C --skip-existing
```

Como el fichero cargado ya contenía todos los identificadores, esta última orden no repitió ninguna consulta. Su finalidad fue producir el JSON y el informe finales con la batería completa como selección de referencia. El resultado consolidado se almacenó en `results/evaluations/bateria_50_20260607_160200.json`.

A.1.3. Ejecución en bloques o por casos

Cuando el servicio presenta una estabilidad menor, la batería también puede dividirse en bloques de cinco o diez ejemplos. El índice comienza en cero:

```
python scripts/evaluate_bateria_50.py --chunk-size 5 --chunk-index 0
```

Los bloques posteriores pueden continuar recuperando el JSON anterior y omitiendo los identificadores ya presentes. También es posible ejecutar únicamente uno o varios casos para revisar un fallo concreto:

```
python scripts/evaluate_bateria_50.py --cases B19_oro_intradia_1wk
python scripts/evaluate_bateria_50.py --cases '
C13_eurusd_informe_intradia C17_nvda_amd_regimenes_riesgo
```

El procedimiento no impide que el modelo genere una respuesta incorrecta, pero reduce el efecto de los fallos operativos. Tras completar cada caso, la herramienta vuelve a escribir el JSON de resultados y el informe Markdown. De este modo, si la ejecución se interrumpe, los casos terminados hasta ese momento permanecen guardados y pueden utilizarse como punto de continuación.

El informe generado se encuentra en `results/reports/evaluacion_bateria_50.md`, con una copia en `docs/evaluacion_bateria_50.md`. Los resultados completos se conservan en `results/evaluations`.

A.2. Configuración de API

Las claves de acceso se guardan localmente en el fichero `.env`, que no debe incorporarse al repositorio. La configuración mínima utilizada con el servidor compatible con OpenAI es:

```
VLLM_BASE_URL=https://servidor-vllm.example/v1
VLLM_API_KEY=...
VLLM_MODEL=openai/gpt-oss-20b
```

La aplicación reconoce tres familias equivalentes de variables:

- las específicas de vLLM: `VLLM_BASE_URL`, `VLLM_API_KEY` y `VLLM_MODEL`;
- las genéricas: `LLM_BASE_URL`, `LLM_API_KEY` y `LLM_MODEL`;
- las compatibles con OpenAI:
`OPENAI_BASE_URL`, `OPENAI_API_KEY` y `OPENAI_MODEL`.

También pueden ajustarse los parámetros de generación y conexión:

```
LLM_TEMPERATURE=0.1
LLM_MAX_TOKENS=4096
LLM_TIMEOUT_SECONDS=120
LLM_VERIFY_SSL=true
```

Si no se especifican estos últimos valores, la aplicación utiliza los mostrados como configuración predeterminada. El proceso que ejecuta el código generado dispone, además, de un límite independiente de 60 segundos. En esta configuración, vLLM describe la infraestructura que sirve el modelo; el modelo evaluado continúa siendo `openai/gpt-oss-20b`, documentado por OpenAI [6].

A.3. Batería de consultas A/B/C

La versión actual de la batería contiene 50 consultas. La distribución concede mayor peso a los niveles B y C porque concentran las tareas con varias métricas, salidas estructuradas, restricciones de formato y mayor probabilidad de error. Los diez casos A representan el 20 % del total; los veinte casos B, el 40 %; y los veinte casos C, el 40 %.

Nivel	Número de casos	Porcentaje
A	10	20 %
B	20	40 %
C	20	40 %

Tabla A.1: Distribución de la batería de 50 consultas.

Los casos reutilizan ocho conjuntos de datos locales: Nvidia, Nvidia y AMD, AAPL, QQQ y SPY, S&P 500, Bitcoin, EUR/USD y oro. En algunos ejemplos se combinan dos fuentes. La tabla siguiente recoge la batería completa mediante su identificador y una descripción resumida. El catálogo digital conserva, además, la consulta literal, las rutas de entrada y las salidas esperadas.

Tabla A.2: Batería completa de 50 consultas por nivel de profundidad.

Identificador	Consulta y objetivo resumidos
A01_nvda_crecimiento_5y	Crecimiento histórico de Nvidia durante cinco años.
A02_nvda_volatilidad_basica_5y	Volatilidad histórica y peor día observado de Nvidia.
A03_nvda_amd_comparacion_simple	Comparación directa del crecimiento de Nvidia y AMD.
A04_aapl_vision_general_3mo	Visión descriptiva de AAPL durante tres meses.
A05_aapl_media_movil_simple	Media móvil de 20 sesiones y comparación con el último cierre.
A06_qqq_spy_retornos_2024	Comparación breve de los retornos de QQQ y SPY en 2024.
A07_sp500_crecimiento_desde_2020	Crecimiento del S&P 500 desde 2020.
A08_btc_crecimiento_2024	Crecimiento de Bitcoin durante 2024.
A09_eurusd_cambio_10d_1h	Variación de EUR/USD en diez días con datos horarios.
A10_oro_rango_1wk_1h	Rango máximo y mínimo del oro durante una semana.
B01_nvda_tabla_evolucion_5y	Análisis ampliado de Nvidia con tabla y serie de evolución.

Continuación de la batería de 50 consultas

Identificador	Consulta y objetivo resumidos
B02_nvda_metricas_riesgo_5y	Retorno y métricas de riesgo de Nvidia durante cinco años.
B03_nvda_resumen_mensual_5y	Mejor y peor mes de Nvidia con resumen mensual.
B04_nvda_amd_tabla_comparativa	Tabla comparativa de rentabilidad, volatilidad y drawdown.
B05_nvda_amd_serie_normalizada	Comparación NVDA/AMD mediante una serie normalizada.
B06_nvda_amd_correlacion	Correlación de retornos y comparación de volatilidad.
B07_aapl_retorno_riesgo_3mo	Relación entre retorno y riesgo de AAPL.
B08_aapl_tecnico_sma_20_50	Medias móviles de 20 y 50 sesiones de AAPL.
B09_aapl_volumen_y_precio	Relación entre volumen y variación del precio de AAPL.
B10_qqq_spy_tabla_serie_2024	Comparación QQQ/SPY con tabla y serie normalizada.
B11_qqq_spy_retorno_riesgo_2024	Retorno, volatilidad y relación retorno-riesgo de QQQ y SPY.
B12_qqq_spy_drawdown_2024	Comparación de drawdown y datos para su evolución temporal.
B13_qqq_spy_meses_2024	Retornos mensuales y activo ganador en cada mes.
B14_btc_2024_perfil_volatil	Perfil de rentabilidad, volatilidad y drawdown de Bitcoin.
B15_btc_2024_mejores_peores_dias	Cinco mejores y cinco peores días de Bitcoin.
B16_sp500_desde_2020_riesgo	Crecimiento, volatilidad y drawdown del S&P 500.
B17_sp500_serie_base_100	Serie base 100 y tabla de rentabilidad anual del S&P 500.
B18_eurusd_intradia_10d	Variación, rango y volatilidad horaria de EUR/USD.
B19_oro_intradia_1wk	Cambio, rango y mejores y peores horas del oro.
B20_btc_sp500_lectura_contextual	Perfil de riesgo de Bitcoin sin incorporar comparaciones externas.
C01_qqq_spy_informe_profesional_2024	Informe QQQ/SPY con tabla, serie normalizada y drawdown.
C02_qqq_spy_formato_cuatro_bloques	Informe con cuatro bloques de salida obligatorios.
C03_qqq_spy_retorno_riesgo_meses	Informe comparativo con retorno, riesgo y ranking mensual.
C04_nvda_amd_ranking_multicriterio	Ranking NVDA/AMD por rentabilidad, riesgo y drawdown.
C05_nvda_amd_informe_cliente	Informe para un usuario no técnico sobre Nvidia y AMD.
C06_nvda_5y_informe_completo	Informe completo de Nvidia con serie base 100 y drawdown.

Continuación de la batería de 50 consultas

Identificador	Consulta y objetivo resumidos
C07_nvda_estres_salida_json	Caso de estrés con cuatro secciones estructuradas.
C08_sp500_informe_desde_2020	Informe detallado del S&P 500 desde 2020.
C09_sp500_crisis_y_recuperacion	Comportamiento anual, caídas y recuperaciones del S&P 500.
C10_btc_2024_informe_riesgo	Informe de riesgo de Bitcoin con extremos y drawdown.
C11_btc_2024_meses_y_drawdown	Análisis mensual y drawdown máximo de Bitcoin.
C12_aapl_informe_tecnico_3mo	Informe técnico de AAPL con medias móviles y volumen.
C13_eurusd_informe_intradia	Informe intradía de EUR/USD con tabla y serie temporal.
C14_oro_informe_intradia	Informe horario del oro con extremos, rango y volatilidad.
C15_caso_control_limitaciones	Caso de control sobre límites y prudencia de la respuesta.
C16_btc_riesgoCola_2024	Riesgo de cola de Bitcoin mediante VaR y expected shortfall.
C17_nvda_amd_regimenes_riesgo	Volatilidad y correlación móviles y cambios de liderazgo.
C18_sp500_episodios_drawdown	Tres mayores episodios de drawdown y sus recuperaciones.
C19_btc_sp500_comparacion_2024	Comparación Bitcoin/S&P 500 sobre fechas comunes.
C20_oro_identidad_instrumento	Tratamiento prudente de GC=F como instrumento concreto.

A.4. Plantillas de prompts de los agentes

Este anexo recoge las partes fijas de las instrucciones utilizadas en las llamadas al modelo. Los bloques indicados entre corchetes representan el contenido variable que se incorpora durante la ejecución: la consulta, la entrada estructurada, las reglas de profundidad, el plan, los resultados o el error correspondiente. Esta separación permite mostrar el contrato estable de cada agente sin reproducir un ejemplo concreto.

A.4.1. Instrucción común

Eres un componente profesional de análisis financiero histórico asistido por LLM. Trabajas exclusivamente con datos históricos ya disponibles en CSV y con el contrato JSON indicado.

No realizas predicciones, asesoramiento de inversión, recomendaciones de compra/venta, análisis fundamental externo

ni incorporas informacion que no este en la entrada.

A.4.2. Agente 1: planificación analítica

Agente 1 - PLANIFICACION ANALITICA.

Devuelve exclusivamente un objeto JSON valido con el esquema indicado. No incluyas Markdown.

Tu tarea es convertir la peticion del usuario en un plan verificable para un script Python posterior. No calcules cifras y no redactes la respuesta final.

Instrucciones obligatorias:

1. Lee la consulta completa y clasifica `analysis_level` como A, B o C usando las reglas incluidas en el payload.
2. Define `analytical_goal` como un objetivo financiero concreto, sin expresiones vagas.
3. Enumera `metrics` con nombres calculables: `retorno_total`, `cagr`, `volatilidad`, `max_drawdown`, `correlacion`, `medias_moviles`, `ranking_mensual` u otros si la consulta los exige.
4. Enumera `required_columns` segun los calculos: `Close` para rentabilidad, `High` y `Low` para rangos, `Volume` para volumen y `Open` cuando sea necesario.
5. Enumera `data_requirements` con la granularidad temporal, los activos, las comparaciones y las validaciones minimas.
6. Enumera `output_requirements` con las tablas, metricas y datos visuales esperados. No pidas imagenes; pide datos JSON.
7. Enumera `presentation_preferences` con el tono y la estructura esperados para el agente interpretador.
8. En `reasoning` justifica brevemente el nivel elegido y las metricas, sin exponer una cadena de pensamiento extensa.

Restricciones: utiliza solo los CSV, tickers y fechas de entrada. No uses noticias, datos fundamentales, precios actuales externos, predicciones ni recomendaciones de inversion.

[ENTRADA ESTRUCTURADA, REGLAS A/B/C Y ESQUEMA JSON]

El contenido variable incluye el esquema de salida y las reglas que distinguen los niveles A, B y C. El agente no recibe una etiqueta impuesta para cada ejemplo: debe deducir el nivel a partir de la consulta y justificar brevemente su elección.

A.4.3. Agente 2: generación de código

Agente 2 - GENERACION DE CODIGO PYTHON.

Devuelve exclusivamente un objeto JSON valido con un unico campo code. No incluyas Markdown.

El valor code debe ser un script Python completo, autocontenido y ejecutable. Debe leer el payload desde argv[1], calcular solo con los CSV indicados y escribir un unico JSON en stdout.

Contrato tecnico obligatorio:

1. Define main() y protege la entrada con
`if __name__ == '__main__':`
2. Lee el payload con
`json.loads(Path(sys.argv[1]).read_text(encoding='utf-8'))`.
No uses open().
3. Usa `load_close_prices` para cierres y `load_market_data` para datos OHLCV. No uses `pd.read_csv` directamente.
4. Usa solo las importaciones permitidas. No uses `red`, `archivos` adicionales, `subprocess`, `os`, `eval`, `exec` ni rutas no recibidas.
5. Devuelve siempre `analysis_level`, `metrics`, `summary` y `limitations`.
6. Para niveles B o C, incluye `table_data` cuando existan comparaciones, `rankings` o bloques de metricas.
7. Para peticiones visuales, incluye `chart_data` o `visualization_data` con tipo sugerido, series muestreadas, ejes y unidades. No generes PNG, PDF o SVG.
8. Ordena por fecha, elimina NaN cuando proceda, evita divisiones por cero y controla datos insuficientes.
9. Pasa la salida por `make_json_safe` antes de `json.dumps`.
10. Si una metrica no puede calcularse, incluye `null` y una limitacion explicita. No inventes valores.

No redactes la respuesta final al usuario. Devuelve solamente resultados estructurados y trazables.

[CONSULTA, ENTRADA VALIDADA, RUTAS, PLAN Y CONTRATO DE SALIDA]

El payload a ade las importaciones permitidas, el contrato de lectura y las funciones comunes de carga. Cuando se solicitan series extensas, el c odigo debe limitar cada una a un m ximo de 120 puntos e indicar las fechas inicial y final. Esta reducci n afecta a la presentaci n del resultado, no a los datos utilizados en los c culos.

A.4.4. Reparaci n del c digo

REPARACION DE CODIGO DEL AGENTE 2.

El codigo anterior fallo o fue rechazado. Devuelve exclusivamente

JSON valido con un unico campo code. El script corregido debe ser completo, no un parche parcial.

Manten el objetivo, el nivel A/B/C y las salidas solicitadas por el plan. Corrige la causa indicada en error_detail y conserva el contrato: Path(sys.argv[1]).read_text, load_close_prices o load_market_data, make_json_safe y un unico JSON en stdout.

No uses open(), pd.read_csv, os, subprocess, eval, exec, red ni Markdown.

[CONSULTA, PLAN, CODIGO ANTERIOR Y DETALLE DEL ERROR]

A.4.5. Agente 3: interpretación

Agente 3 - INTERPRETACION FINANCIERA EN ESPANOL.

Redacta la respuesta final para el usuario usando unicamente execution_output y analysis_plan. No recalculas, no completes cifras ausentes y no incorpores datos externos.

Devuelve texto en espanol con frases completas. No devuelvas JSON, diccionarios, bloques de codigo ni una repeticion aislada de las metricas.

Reglas de respuesta:

1. Ajusta la extension al analysis_level: A breve y directo; B estructurado con las metricas principales; C como informe compacto con apartados claros.
2. Distingue datos historicos observados, lectura interpretativa y limitaciones.
3. Si existen tablas o datos visuales, explica que muestran sin fingir que se ha generado una imagen.
4. Si una metrica es null o hay datos insuficientes, indicalo como una limitacion.
5. No uses tono persuasivo, no recomiendes comprar o vender y no predigas rendimientos futuros.
6. Manten un espanol claro, profesional y comprensible para un usuario no tecnico.

[PLAN ANALITICO Y RESULTADOS CALCULADOS]

A.5. Trazabilidad de una ejecución

Cada caso conserva información suficiente para reconstruir el recorrido completo. El informe de evaluación presenta la entrada utilizada, el estado final, el tiempo observado, la respuesta almacenada en `state.final_answer`, la salida estructurada de la ejecución y las métricas cualitativas. El fichero JSON mantiene, además, el plan analítico, el código generado, los avisos, el mensaje de error y las rutas de los artefactos de ejecución.

Elemento conservado	Utilidad durante la revisión
Entrada de la consulta	Permite comprobar activos, periodo, intervalo y datos disponibles.
Plan analítico	Muestra cómo interpretó la petición el primer agente.
Código generado	Permite revisar cómo se trasladó el plan a operaciones calculables.
Salida de ejecución	Contiene las métricas, tablas, series y limitaciones obtenidas.
Respuesta final	Permite contrastar la redacción con los resultados calculados.
Estado, avisos y errores	Identifican el punto de finalización y los intentos de reparación.
Tiempo y artefactos	Ayudan a reconstruir la ejecución y localizar sus registros.

Tabla A.3: Información conservada para revisar cada caso de la batería.

La salida legible se almacena en `docs/evaluacion_bateria_50.md`; el catálogo de entrada se encuentra en `data/catalog/bateria_50_ejemplos.json`; y los resultados completos se conservan en `results/evaluations`. El informe Markdown limita la extensión de algunas estructuras para mantener la legibilidad, mientras que el JSON conserva el detalle íntegro.

A.6. Rúbrica de evaluación cualitativa

La evaluación utiliza cinco métricas cualitativas puntuadas de 0 a 2: el 0 señala un incumplimiento o un fallo esencial; el 1, un cumplimiento parcial; y el 2, un resultado adecuado. La suma ofrece una referencia sobre 10 puntos. La valoración inicial es heurística, puede contrastarse con los artefactos conservados y sirve para priorizar la revisión, pero no sustituye una evaluación humana detallada.

Métrica	Aspecto evaluado
Comprensión de la consulta	Correspondencia del plan con los activos, el periodo, el nivel y las restricciones solicitadas.
Pertinencia de las métricas	Adecuación y suficiencia de las medidas elegidas para responder a la petición.
Calidad de la ejecución analítica	Corrección del cálculo, uso de los datos adecuados y presencia de una salida estructurada.
Fidelidad de la interpretación	Coherencia de la respuesta final con el plan y los resultados ejecutados.
Claridad, utilidad y prudencia	Calidad comunicativa, reconocimiento de límites y ausencia de predicciones o recomendaciones.

Tabla A.4: Métricas cualitativas aplicadas a los casos de evaluación.